

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Simplified Cross-Platform HRM for Puzzle-Solving

By
Fionn McCarthy

April 26, 2026

Minor Dissertation

**Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.**

Contents

1	Introduction	2
1.1	Context	2
1.2	Project Overview	3
1.3	Objectives	4
1.4	Dissertation Structure	5
2	Methodology	6
2.1	Research Methodology	6
2.2	Software Development Methodology	7
2.2.1	Code Review and Quality Assurance	7
2.2.2	Iterative Architecture Correction	8
2.2.3	Work Packages and Division of Labour	8
2.2.4	Version Control and Repository Structure	8
2.3	Experimental Methodology	9
2.3.1	Independent and Dependent Variables	10
2.3.2	Task Descriptions	10
2.3.3	Training Protocol	11
2.3.4	Evaluation Protocol	11
2.4	Ethical Considerations	12
2.5	Tools and Technologies	12
3	Technology Review	13
3.1	Chain-of-Thought Reasoning and Its Limitations	13
3.2	HRM Architecture and Mechanisms	13
3.2.1	Core Design Principles	13
3.2.2	Hierarchical Convergence	14
3.2.3	Fixed-Point Iteration Requirements	15
3.2.4	Memory Patterns	15
3.2.5	Training Methodology: One-Step Gradient Approximation	15
3.3	Performance and Benchmarks	16
3.3.1	Benchmark Tasks	16
3.3.2	Empirical Results	16
3.4	Comparative Analysis and Critical Perspectives	17
3.4.1	Advantages Over Chain-of-Thought Reasoning	17
3.4.2	Limitations and Trade-offs	17
3.4.3	Recent Critical Findings	18
3.5	Biological Correspondence	18
3.6	Cross-Platform Inference Frameworks	19
3.6.1	ONNX Representation Challenges	19

3.6.2	Runtime Optimisation Applicability	19
3.7	Edge Device Optimisation Techniques	20
3.7.1	Quantisation for Iterative Architectures	20
3.7.2	ARM-Specific Optimisations	21
3.8	Research Gaps and Summary	21
4	System Design	23
4.1	Overall System Architecture	23
4.2	Model Architecture: Full HRM	24
4.2.1	Input Network	24
4.2.2	Recurrent Modules	24
4.2.3	Output Network	25
4.2.4	Training with One-Step Gradient Approximation	25
4.3	Model Architecture: L-Module Only	25
4.4	Task Implementation: Sudoku	26
4.4.1	Input Encoding	26
4.4.2	Output Representation	26
4.4.3	Dataset Generation	27
4.5	Task Implementation: Weighted Maze	27
4.5.1	Input Encoding	27
4.5.2	Output Representation	28
4.5.3	Dataset Generation	28
4.6	Training Pipeline	29
4.7	Evaluation Pipeline	29
4.8	Trajectory Visualiser	30
4.9	Design Decisions and Trade-offs	31
5	System Evaluation	32
5.1	Experimental Setup	32
5.2	Sudoku Results	32
5.2.1	Analysis	33
5.3	Weighted Maze Results	34
5.3.1	Prediction Visualisation	35
5.3.2	Trajectory Visualisation	36
5.3.3	Analysis	37
5.4	Edge Deployment on Raspberry Pi 5	38
5.5	Cross-Task Comparison	40
5.6	Evaluation Against Objectives	40
5.7	Discussion	41
5.7.1	Why the L-Module Only Was Prioritised	41
5.7.2	Strengths	41
5.7.3	Limitations	41
5.8	Further Benchmarking	42
6	Conclusion	43
6.1	Summary	43
6.2	Key Findings	43
6.3	Objectives Revisited	44
6.4	Limitations	44

6.5	Future Work	44
6.6	Final Remarks	45

List of Figures

2.1	Repository commit history showing the four development phases and the distribution of contributions between both team members across November 2025 to March 2026.	7
2.2	Project roadmap showing the timeline of work packages from October 2025 to April 2026.	10
3.1	Forward residual comparison across architectures. Left: HRM H-module (red) and L-module (blue) maintain sustained activity across hundreds of steps. Middle: Standard RNN with rapid residual decay within tens of steps. Right: Deep Neural Network with fixed depth. Reproduced from Wang <i>et al.</i> [1].	14
3.2	The HRM dual-module architecture as described by Wang <i>et al.</i> [1]. The input is embedded and fed to the L-module, which iterates for N steps per cycle. After each cycle, the H-module updates using the converged L-module state, resetting the L-module’s computational space for the next cycle. The output head produces the final prediction after all cycles complete. Modules coloured in blue experience one-step gradient approximation during training, greatly reducing memory requirements from $O(n)$ with BPTT to $O(1)$ with one-step gradient approximation.	16
4.1	High-level system architecture showing the pipeline from dataset generation through model training to evaluation across multiple hardware targets. . .	24
4.2	Training pipeline showing the flow from data loading through iterative refinement to gradient update using one-step gradient approximation with deep supervision.	29
4.3	Trajectory visualiser output for a 15×15 weighted maze, showing the model’s prediction at each of 16 reasoning steps as a colour-coded grid, allowing for iterative refinement behaviour to be inspected directly.	30
5.1	Sudoku 9×9 training curves. Top left: training and validation loss decreasing over epochs. Top right: token accuracy rising to approximately 96% on the validation set. Bottom left: puzzle accuracy climbing to approximately 70% on the validation set during this particular training run. Bottom right: average L2 residual across reasoning steps, showing the model’s internal convergence behaviour.	34
5.2	Evaluation script output for a single training run on 1,000 test puzzles, yielding 96.2% token accuracy and 87.9% puzzle accuracy. An example solved puzzle is displayed with given clues in bold, correct predictions in green, and incorrect predictions in red.	35

5.3	Weighted Maze 15×15 training curves. Top left: training loss. Top right: token accuracy. Bottom left: puzzle accuracy. Bottom right: average L2 residual.	36
5.4	Evaluation script output for a single test run on 1,000 test mazes, showing 98.0% cell accuracy and 76.4% puzzle accuracy. Below the summary, an example of the model’s correct prediction is displayed.	37
5.5	Correctly solved 15×15 weighted maze (Maze #10). The model achieves 100% cell accuracy and 100% path accuracy. Left: model prediction. Right: ground truth. The colour legend distinguishes walls (black), floor (white), weighted cells (brown tones), correct path (green), start (blue), and goal (red).	37
5.6	Incorrectly solved 15×15 weighted maze (Maze #3). The model achieves 97.3% cell accuracy but only 90.3% path accuracy. Red cells indicate false positives (cells predicted as on-path that are not), and purple cells indicate missed path segments. The errors occur near the bottom of the maze where the model selects a suboptimal route through lower-cost cells.	38
5.7	Reasoning trajectory for Maze #10 across 16 iterative steps. At Step 1, the model achieves only 86% cell accuracy and 36% path accuracy. By Step 5, it converges to 100% on both metrics, with $\Delta 0$ changed cells in subsequent steps. The visualisation demonstrates how the model progressively refines its solution through iterative latent reasoning, with the bulk of computation occurring in the first four to five steps.	39

List of Tables

2.1	Division of work packages between team members.	9
2.2	Tools and technologies used in the project.	12
3.1	Comparison of HRM and Chain-of-Thought LLMs across key dimensions. .	17
3.2	Critical research gaps in HRM edge deployment.	22
4.1	Repository directory structure.	23
4.2	Weighted Maze input token encoding.	28
5.1	Training hyperparameters used for both tasks.	33
5.2	L-module-only performance on mixed-difficulty 9×9 Sudoku, averaged across 10 seeds.	33
5.3	L-module-only performance on 15×15 Weighted Maze, averaged across 10 seeds.	35
5.4	L-module-only performance summary across tasks, averaged across 10 seeds.	40

Chapter 1

Introduction

1.1 Context

Reasoning is the ability to plan and execute complex, goal-oriented actions in sequence, and remains one of the core challenges of artificial intelligence. Contemporary Large Language Models (LLMs) such as GPT-5.4, Claude Opus 4.6, and DeepSeek-V3.2 approach reasoning through Chain-of-Thought (CoT) reasoning, a technique in which the model allocates intermediate tokens for “thinking” before producing a final answer [2]. While CoT has improved performance on many benchmarks, it suffers from core limitations. Errors in early steps cascade through the reasoning chain [3, 4]. Problems that require backtracking or parallel exploration become intractable, and the fixed-depth Transformer architecture constrains these models to complexity classes that cannot solve polynomial-time problems [5]. These are not merely theoretical concerns: state-of-the-art CoT models as of August 2025 achieve 0% accuracy on expert-level Sudoku puzzles that demand systematic backtracking [1, 6].

The Hierarchical Reasoning Model (HRM), introduced by Wang *et al.* [1] at Sapient Inc., offers an alternative approach. Inspired by hierarchical processing in the mammalian brain, HRM operates in continuous latent space through two coupled recurrent modules: a high-level (H) module responsible for slow, abstract planning, and a low-level (L) module that performs rapid, detailed computation. Rather than generating tokens sequentially, the model iteratively refines a latent representation of the solution until convergence. With only 27 million parameters and 1,000 training examples, HRM achieves 40.3% accuracy on ARC-AGI-1, outperforming models with hundreds of billions of parameters, and solves 55% of expert-difficulty Sudoku puzzles where all tested CoT models fail entirely [1].

However, subsequent work by Ge *et al.* [7], corroborated by the ARC Foundation [8], suggests that HRM’s success may derive primarily from computational depth and iterative refinement rather than from the hierarchical structure itself. An L-module-only variant achieves comparable accuracy while training significantly faster, motivating investigation of whether the simpler architecture suffices. These findings are discussed in detail in Chapter 3.

Despite its compact size, HRM has been evaluated only on a narrow set of grid-based tasks with uniform costs: Sudoku, mazes, and ARC puzzles. No published work has examined how HRM performs on tasks with heterogeneous traversal costs. Furthermore, the official Sapient Inc. implementation is tightly coupled to CUDA and NVIDIA GPUs, leaving no platform-agnostic implementation available for educational or experimental use, and no empirical data on deployment to resource-constrained edge devices.

1.2 Project Overview

This project presents an independent, cross-platform implementation of a single-module variant of the Hierarchical Reasoning Model, developed as a pair project by Fionn McCarthy and Kyrylo Kozlovskiy. The implementation is based on the architecture described by Wang *et al.* [1] and the official Sapien Inc. codebase¹, with several extensions. The simplified model was implemented from scratch, deliberately designed for cross-platform execution across GPU, CPU, and ARM platforms. The architectural details and rationale for the design are described in Chapter 4.

Alongside the reimplementing, the project introduces a Weighted Maze task in which each traversable cell in a 15×15 grid carries an associated movement cost, requiring the model to find a path that minimises total cost rather than step count. Ground-truth solutions are computed via Dijkstra’s algorithm. This task has not previously been evaluated with any HRM variant. Additionally, the project investigates deployment of the trained model on a Raspberry Pi 5 to assess the viability of HRM inference on edge hardware.

The contributions of this project are as follows:

1. An L-module-only variant implemented from scratch in a platform-agnostic codebase, enabling the same model to train and infer on GPU, CPU, and ARM hardware without modification.
2. A Weighted Maze task with a generator that produces puzzles of configurable size and cost distribution, and a solver that computes optimal reference paths using Dijkstra’s algorithm.
3. A dataset pipeline that standardises encoding and formatting across both Sudoku and Weighted Maze tasks, allowing the same training and evaluation infrastructure to be reused across domains.
4. A model trajectory visualiser for inspecting iterative refinement behaviour during inference, providing a qualitative window into how the latent state evolves across reasoning steps.
5. A training performance tracking system that logs metrics to CSV across epochs and automatically generates loss and accuracy plots.
6. Export of trained models to ONNX format and deployment on a Raspberry Pi 5, providing empirical data on HRM inference performance on ARM-based edge hardware.

My individual contributions to this project focus on:

1. Leading the research into HRM architecture and the internal mechanisms that informed the implementation.
2. Serving as lead developer of the full HRM model, and implementing the one-step backpropagation training methodology in the L-module-only variant.
3. Leading the research and design of the Weighted Maze puzzle task, including the dataset generator and optimal path computation via Dijkstra’s algorithm.

¹<https://github.com/sapieninc/HRM/>

4. Building the Sudoku dataset generator and standardising the encoding and formatting conventions used across all datasets.
5. Implementing the model trajectory visualiser for inspecting iterative refinement behaviour during inference.
6. Developing the training performance tracking infrastructure, including per-epoch logging to CSV and automatic generation of loss and accuracy plots.

Kyrylo Kozlovskiy led the implementation of the L-module-only variant, the continuous integration pipeline, the ONNX export and zero-dependency inference scripts, and the Raspberry Pi 5 edge deployment and benchmarking.

The project has been developed since October 2025 and is publicly available at <https://github.com/fionntmcc/cross-platform-hrm>.

1.3 Objectives

The project pursues the following four objectives:

1. **Implement a functional Hierarchical Reasoning Model.** Build a working HRM with both the H-module and L-module, capable of training from scratch and performing inference on grid-based reasoning tasks, using a pure PyTorch codebase that avoids CUDA-specific extensions.
2. **Implement an L-module-only variant.** Construct a variant of the HRM that removes the high-level module entirely, preserving only the low-level recurrent module with eight weight-shared transformer blocks and 16 iterative reasoning steps, in order to investigate whether hierarchical structure is necessary for effective latent reasoning.
3. **Train and benchmark the L-module-only variant on Sudoku puzzles.** Train the L-module-only variant on 9×9 Sudoku puzzles of mixed difficulty, ranging from easy to expert-level puzzles that require substantial backtracking, and evaluate accuracy and convergence behaviour. Comparison with the full HRM is a secondary goal.
4. **Train and benchmark the L-module-only variant on a Weighted Maze task.** Design and implement a Weighted Maze puzzle generator in which cells have heterogeneous traversal costs, train the L-module-only variant on this task, and evaluate whether the HRM architecture can generalise to cost-sensitive pathfinding. A target accuracy of 60% on 15×15 mazes was agreed upon as a sufficient benchmark for success. Comparison with the full HRM is a secondary goal.

Additionally, the project explores the feasibility of deploying the trained models on edge devices, specifically a Raspberry Pi 5, to assess inference latency and accuracy in resource-constrained environments.

1.4 Dissertation Structure

The remainder of this dissertation is organised as follows. Chapter 2 describes the research and software development methodologies employed throughout the project, including the Agile workflow, code review process, and division of work between team members. Chapter 3 presents a technology review synthesising the literature on HRM architecture, CoT reasoning, critical perspectives on hierarchical structure, energy implications, and edge deployment considerations. Chapter 4 details the system design, covering the architecture of both model variants, the Sudoku and Weighted Maze task implementations, the dataset generators, and the training and evaluation pipelines. Chapter 5 evaluates the system against the objectives set out above, presenting benchmark results and a critical discussion of strengths and limitations. Finally, Chapter 6 summarises the findings and identifies directions for future work.

Chapter 2

Methodology

This chapter describes the research methodology, the software development methodology, and the experimental methodology used throughout the project. It also outlines the tools, hardware, and division of work between both team members.

2.1 Research Methodology

The research component of this project follows a structured literature review approach guided by three questions: (1) What architectural mechanisms enable HRM’s performance on definitive reasoning tasks? (2) How does the model compare empirically and theoretically to Chain-of-Thought (CoT) approaches? (3) What limitations and open questions remain regarding HRM’s design and applicability?

Sources were identified through academic databases including IEEE Xplore, Scopus, arXiv, and ACM Digital Library, supplemented with technical reports from AI research organisations. Given the recent emergence of the HRM, the review includes preprints alongside peer-reviewed publications. Inclusion criteria required sources to address recurrent or novel reasoning architectures, provide empirical or theoretical comparisons to Transformer-based reasoning models, or evaluate definitive reasoning tasks. Sources published between 2021 and 2025 were prioritised, with earlier works included where they were foundational or supplementary in purpose. Sources were excluded if they addressed AI without reasoning-specific evaluation or lacked sufficient methodological detail.

Both team members conducted independent literature reviews prior to the implementation phase. My review covered 23 sources and focused on HRM architecture, performance benchmarks, comparative analysis with CoT LLMs, critical perspectives from Ge *et al.* [7], biological correspondence, and energy implications. Kyrylo Kozlovskyi’s review focused on deployment considerations for edge devices, including inference on hardware such as the Raspberry Pi. These two reviews are synthesised in Chapter 3.

A primary limitation of the research methodology is the premature state of HRM literature. Only two substantial technical papers directly address the architecture [1, 7], with limited independent replication. This constraint necessitated drawing on related work from adjacent fields, including Deep Equilibrium Models [9], Adaptive Computation Time [10], and recurrent depth approaches [11], to apply context to findings.

2.2 Software Development Methodology

The project followed an Agile development methodology with iterative sprints, beginning in October 2025. Work was organised using a Kanban board within GitHub Projects, providing visibility over task progress across both team members. Each unit of work was captured as a GitHub Issue, and development followed a branch-per-issue workflow: for every issue, a dedicated branch was created, worked on, and merged back into the main branch via a Pull Request (PR) upon completion. Figure 2.1 shows the resulting commit history across both contributors over the lifetime of the project.

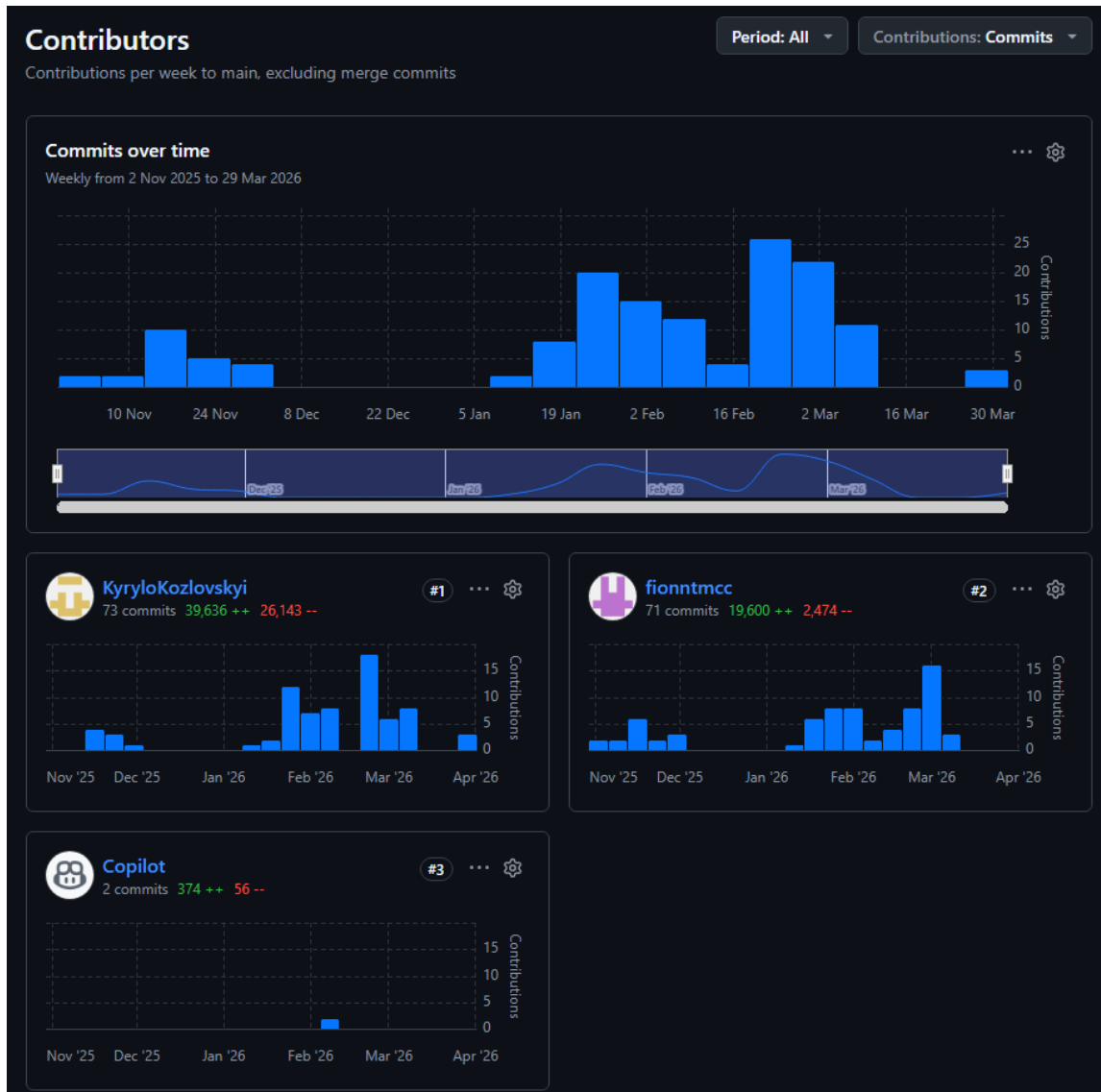


Figure 2.1: Repository commit history showing the four development phases and the distribution of contributions between both team members across November 2025 to March 2026.

2.2.1 Code Review and Quality Assurance

A code review process was followed throughout the project. Every Pull Request required review before merging. I reviewed Kyrylo Kozlovskiy's PRs and he reviewed mine, en-

sureing that both team members maintained familiarity with the full codebase and that no code entered the main branch without scrutiny. In addition to human review, GitHub Copilot was configured to automatically review every Pull Request, providing an additional layer of automated feedback on code quality and potential bugs.

Kyrylo Kozlovskyi implemented a CI pipeline on GitHub which enforced Black and Ruff linters, as well as running pytest scripts upon commit. This ensured a consistent code structure and quality in the codebase.

2.2.2 Iterative Architecture Correction

An important methodological note concerns the evolution of the model architecture during the early stages of development. As lead developer of the Full HRM, I built an initial prototype in November 2025 that adopted HRM-inspired naming and module structure but did not faithfully reproduce the iterative reasoning loop described by Wang *et al.* [1]. The prototype stacked multiple Worker modules sequentially rather than applying a single weight-shared module iteratively, which collapsed the intended fixed-point iteration into a deeper feedforward computation.

The error was identified through close re-reading of the Ge *et al.* [7] ablation paper, which clarified that the “8-layer L-module” described in the original work refers to the internal transformer depth of a single weight-shared module applied iteratively, rather than eight separately stacked modules. Correcting this distinction reshaped subsequent development of both the Full HRM and the L-module-only variant, and is documented here as a genuine methodological learning outcome about the importance of precise terminology when reproducing recent research architectures.

2.2.3 Work Packages and Division of Labour

Work was divided into discrete work packages assigned to each team member based on their areas of focus. Table 2.1 summarises the primary responsibilities, with specific components named where appropriate.

While primary responsibilities were assigned as shown, both team members contributed across all areas through the code review process and collaborative debugging.

2.2.4 Version Control and Repository Structure

The project is hosted on GitHub at <https://github.com/fionntmcc/cross-platform-hrm>. The repository is structured as a Python package under the `hrm/` directory, with separate directories for tests (`tests/`), documentation (`docs/`), scripts (`scripts/`), and CI/CD workflows (`.github/workflows/`). A GitHub Actions pipeline was configured to run automated tests on each push, ensuring that changes did not introduce regressions.

The repository contains 161 commits at the time of writing, spanning from the initial commit in November 2025 through to the final ONNX inference work in March 2026. Development activity clustered into four distinct phases: project setup and 4×4 prototyping in November 2025, core HRM module implementation through January 2026, the L-module-only variant alongside training infrastructure and CI/CD in February 2026, and Weighted Maze integration with ONNX export and metrics logging in March 2026. Figure 2.2 provides an overview of the project timeline, mapping each work package to its planned and actual delivery window across the academic year.

Work Package	Fionn McCarthy	Kyrylo Kozlovskiy
WP1: Literature Review	HRM architecture, benchmarks, CoT comparison, critical perspectives, biological correspondence, energy implications	Edge deployment, ONNX, ARM hardware, quantisation, Raspberry Pi 5
WP2: Core Architecture	Lead developer of Full HRM, Worker module with gating, halting mechanism (ACT), outer iteration loop, transformer components, UnifiedHRM assembly	L-module-only (Simplified HRM) variant, project setup, RMSNorm, InputNetwork, PlannerModule, OutputNetwork, fixed-point iteration loop
WP3: Dataset Generation	4×4 and 9×9 Sudoku generators with difficulty levels, heuristic maze generator, Weighted Maze generator with Dijkstra solver, encoding and formatting standardisation	SudokuDataset PyTorch class, validation utilities, dataset export/import CLI (JSON/CSV)
WP4: Training and Evaluation	Training pipeline design, metrics and logger modules, trajectory visualiser, multi-seed analysis, benchmark collection	Training run execution, maze visualiser, demo and run scripts
WP5: Deployment	Vocab parameterisation for multi-task support	ONNX export pipeline, zero-dependency inference script, Raspberry Pi 5 deployment and benchmarking
Infrastructure	README	GitHub Actions CI (Black, Ruff, pytest), pre-commit hooks, GitHub Releases registry, model publish script
WP6: Dissertation	Introduction, Methodology, Technology Review, Design	Evaluation, Conclusion, formatting

Table 2.1: Division of work packages between team members.

2.3 Experimental Methodology

The experimental methodology was designed to enable a controlled comparison between the full HRM (with both H-module and L-module) and the L-module-only variant across two reasoning tasks: Sudoku and Weighted Maze.

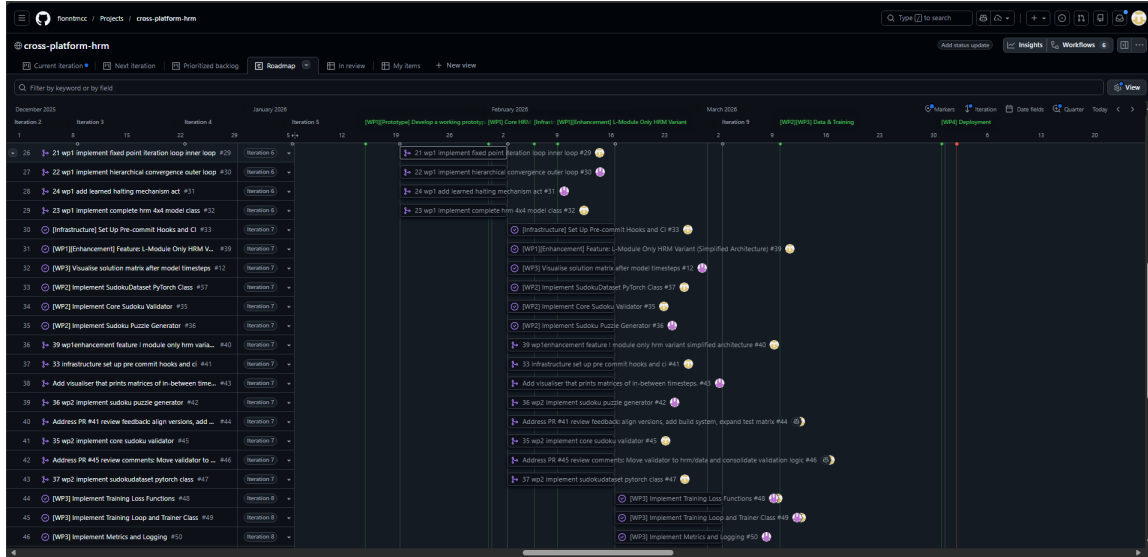


Figure 2.2: Project roadmap showing the timeline of work packages from October 2025 to April 2026.

2.3.1 Independent and Dependent Variables

The primary independent variables across all experiments are the model architecture (full HRM versus L-module-only), the task (Sudoku versus Weighted Maze), and the inference platform (GPU, CPU, Raspberry Pi 5). For each task, both variants were trained using identical hyperparameters, datasets, and training procedures, isolating the effect of the hierarchical structure on performance.

The dependent variables measured are:

- **Puzzle Accuracy:** the percentage of test puzzles solved with every cell correct.
- **Token/Cell Accuracy:** the percentage of individual cells predicted correctly.
- **Training time:** wall-clock time to reach convergence or a fixed number of epochs.
- **Convergence behaviour:** training and validation loss curves over epochs.
- **Inference latency:** time per puzzle at inference, measured across GPU, CPU, and Raspberry Pi hardware.
- **Generalisation:** accuracy on unseen data generated with different random seeds, particularly relevant for the Weighted Maze task.

2.3.2 Task Descriptions

Sudoku. The Sudoku task follows the protocol established by Wang *et al.* [1]. A custom generator produces puzzles ranging from easy (fewer than 5 backtracks required) to expert-level (50+ backtracks), matching the difficulty distribution of the Sudoku-Extreme dataset used in the official HRM paper [12]. The model receives a partially filled 9×9 grid as input and must produce the completed grid as output. The custom dataset generator was developed to produce training and evaluation data matching the requirements of the Sudoku-Extreme protocol.

Weighted Maze. The Weighted Maze task is a novel contribution of this project. Each puzzle consists of a 15×15 grid in which traversable cells are assigned a movement cost, drawn as an integer weight from 1 to 9. The objective is to find the path from a designated start cell to a goal cell that minimises the total traversal cost. Ground-truth optimal paths are computed using Dijkstra’s algorithm. This differs from the Maze-Hard task used in the original HRM evaluation [1], which uses uniform movement costs and evaluates only whether the model finds the shortest path by step count. The Weighted Maze requires the model to reason about cumulative cost, introducing a layer of complexity not previously tested with any HRM variant. A custom dataset generator was developed to produce training and evaluation data for this task.

2.3.3 Training Protocol

Both model variants were trained from random initialisation using the same training pipeline. Training was conducted on a single NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12 GB GDDR7. The PyTorch framework was used without additional CUDA extensions such as FlashAttention, as the project prioritised cross-platform compatibility over raw training speed. This decision was deliberate: one of the project’s goals is to explore whether HRM can be trained and deployed outside of high-end GPU environments. Mixed-precision training (AMP) was enabled throughout.

Hyperparameters were kept consistent between the full HRM and L-module-only variant for each task to ensure a fair comparison. Training used a batch size of 512 for the 9×9 Sudoku task (consuming approximately 4.1 GB of VRAM) and a reduced batch size of 128 to 256 for the 15×15 Weighted Maze task due to the larger grid. The learning rate was $3e-4$ with weight decay of 0.1, and training ran for up to 200 epochs per task. Specific hyperparameter values for each experiment are reported alongside results in Chapter 5. Training progress was monitored through loss curves and periodic evaluation on held-out test sets.

2.3.4 Evaluation Protocol

Evaluation was performed on held-out test sets not seen during training. For Sudoku, accuracy was measured at both the puzzle level (every cell correctly filled) and the token level (percentage of individual cells correct). For Weighted Maze, accuracy was measured as the percentage of puzzles in which the model produced the optimal minimum-cost path, with cell-level accuracy also reported. Maze generalisation was assessed by evaluating on datasets generated with different random seeds, producing layouts the model had never encountered during training. In addition to accuracy, training time and inference latency were recorded to assess practical efficiency.

Final accuracy figures were computed by averaging results across 10 independent runs, each using a different random seed and evaluating on 1,000 test puzzles, to account for variance in training outcomes.

Inference was benchmarked across three hardware configurations to evaluate the model’s portability:

- **GPU:** NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12 GB GDDR7 (my laptop).
- **CPU:** Intel i7-12700H @ 4.7GHz processor without GPU acceleration (Kyrylo Kozlovskyi’s laptop).

- **Edge device:** Raspberry Pi 5 with 16 GB RAM, running inference only.

This range of hardware allows the project to assess whether HRM’s compact architecture translates into practical deployability beyond conventional GPU setups.

2.4 Ethical Considerations

This project does not involve human participants, personal data, or sensitive information. All datasets are synthetically generated (Sudoku puzzles and Weighted Maze grids) and contain no personally identifiable information. The project uses publicly available open-source software and builds upon research published under open-access licences. No ethical approval was required.

2.5 Tools and Technologies

Table 2.2 summarises the key tools and technologies used throughout the project.

Tool / Technology	Purpose
Python 3	Implementation language
PyTorch	Deep learning framework for model implementation and training
NumPy	Dataset generation, storage, and numerical operations
matplotlib	Training curve visualisation and performance plotting
GitHub	Version control, issue tracking, project management, code review
GitHub Actions	Continuous integration and automated testing
GitHub Copilot	Automated Pull Request review
L ^A T _E X	Dissertation typesetting
Raspberry Pi 5 (16 GB)	Edge device for inference testing
NVIDIA RTX 5070 Ti Laptop GPU (12 GB)	GPU for model training
ONNX	Cross-platform model export for portable inference

Table 2.2: Tools and technologies used in the project.

Chapter 3

Technology Review

3.1 Chain-of-Thought Reasoning and Its Limitations

Contemporary Large Language Models tackle complex problems through Chain-of-Thought (CoT) reasoning [2], where sequential tokens are generated prior to the final response. While this makes their logic transparent and sometimes effective, it suffers from brittleness. One wrong step can cascade through the entire chain [3, 4], and problems requiring parallel exploration or backtracking become computationally intractable. According to the International Energy Agency, LLMs accounted for 415 TWh of energy, or 1.5% of global energy usage as of January 2025 [13].

Furthermore, the fixed-depth Transformer architecture constrains these models to complexity classes that cannot handle polynomial-time problems efficiently [5]. This concern reflects in benchmarks on more complex puzzles: state-of-the-art CoT models including GPT o3-mini-high, Claude 3.7, and DeepSeek R1 achieve 0% accuracy on expert-level Sudoku puzzles that demand systematic backtracking [1, 6]. Analysis of scaling behaviour reveals that increasing Transformer width at fixed depth yields no improvement on Sudoku-Extreme, while increasing depth saturates quickly due to training instabilities [1]. These findings suggest that effective computational depth, rather than raw parameter count, is the primary bottleneck for complex reasoning.

3.2 HRM Architecture and Mechanisms

The HRM implements a recurrent architecture that addresses the computational depth limitations of fixed-depth Transformers while avoiding training instabilities common in Recurrent Neural Networks (RNNs) [14, 11], such as early convergence and exploding and vanishing gradients in Backpropagation Through Time (BPTT) [1, 15].

3.2.1 Core Design Principles

The HRM’s architecture is inspired by three fundamental principles observed in mammalian cortical processing [1, 16]. First, *hierarchical processing* organises computation across multiple levels, where higher-level areas integrate information over extended timescales while lower-level areas handle immediate, detailed processing. Second, *temporal separation* establishes distinct operational timescales analogous to neural rhythms: slow theta waves (4–8 Hz) for high-level integration and fast gamma oscillations (30–100 Hz) for local computation. Third, *recurrent connectivity* enables iterative refinement through feedback

loops, allowing progressive error correction without the computational burden of BPTT [15, 17].

The model consists of four components: an input network $f_I(\cdot; \theta_I)$ that converts inputs into a working representation, a low-level recurrent module $f_L(\cdot; \theta_L)$, a high-level recurrent module $f_H(\cdot; \theta_H)$, and an output network $f_O(\cdot; \theta_O)$ that generates predictions. The computation unfolds over N high-level cycles, each consisting of T low-level timesteps, yielding $N \times T$ total computational steps. The L-module updates at every timestep conditioned on its previous state, the current H-module state, and the input representation, while the H-module updates only once per cycle using the converged state of the L-module [1].

3.2.2 Hierarchical Convergence

As described by Geng *et al.* [14], a major challenge in recurrent architectures is premature convergence. As hidden states approach fixed points, update size diminishes, halting computation and limiting effective reasoning depth. Standard RNNs display this behaviour within tens of iterations.

HRM addresses this through hierarchical convergence, maintaining computational activity across hundreds of steps. During each high-level cycle, the L-module performs T iterative updates, converging to a local equilibrium determined by the current H-module state. Upon convergence, the H-module incorporates the L-module’s final state and performs a single update, establishing a new context that effectively “resets” the L-module’s computational space and initiates convergence toward a different equilibrium in the subsequent cycle.

This hierarchical structure yields effective computational depth of $N \times T$ rather than T typical of standard RNNs. Empirical analysis demonstrates this advantage: as shown in Figure 3.1, forward residuals in standard RNNs decay toward zero within 20–30 steps, while HRM maintains computational activity for 200+ steps [1].

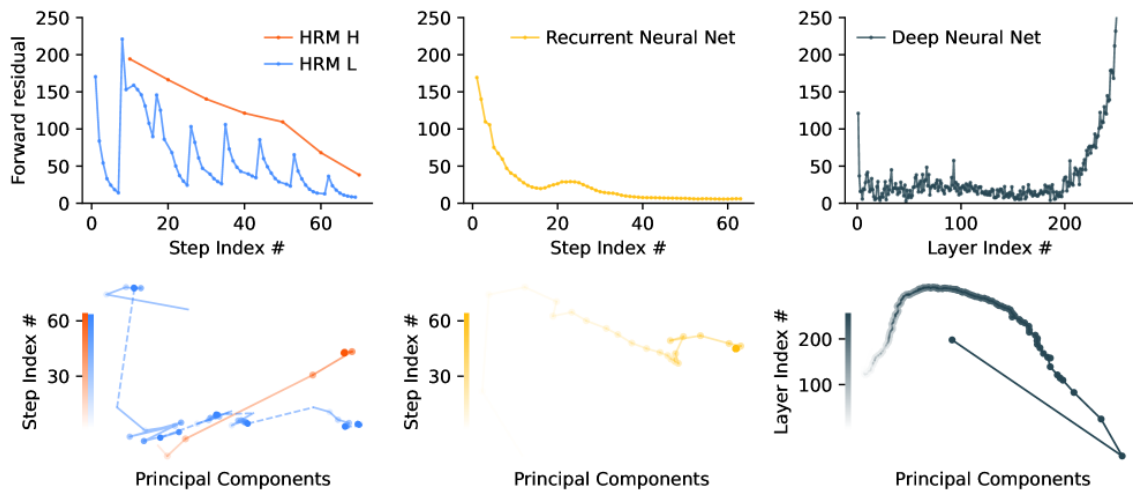


Figure 3.1: Forward residual comparison across architectures. Left: HRM H-module (red) and L-module (blue) maintain sustained activity across hundreds of steps. Middle: Standard RNN with rapid residual decay within tens of steps. Right: Deep Neural Network with fixed depth. Reproduced from Wang *et al.* [1].

3.2.3 Fixed-Point Iteration Requirements

The computational pattern underlying HRM differs from the single-pass inference of feed-forward architectures. HRM employs fixed-point iteration where the low-level module repeatedly applies a transformation until convergence [1], enabling the computational depth necessary for complex reasoning tasks with definitive solutions.

The number of iterations required for convergence varies substantially based on problem complexity and input characteristics. Deep equilibrium model theory demonstrates that speed of convergence depends on the spectral properties of the recurrent operator’s Jacobian [9]: well-conditioned problems converge quickly while less clearly defined problem spaces require extensive iteration. The problem complexity determines iteration counts ranging from dozens to hundreds [1], leading to a wide range of latency during inference.

The iterative nature prevents certain optimisations effective for feedforward networks: operator fusion strategies that merge consecutive layers become difficult when the same operators execute repeatedly with varying inputs, and static memory allocation based on predetermined layer sequences becomes problematic when iteration counts vary dynamically [18].

3.2.4 Memory Patterns

HRMs exhibit constant memory consumption regardless of reasoning depth, a key advantage over Transformers where memory scales linearly with sequence length [9]. While Transformers may exhaust device memory on extended reasoning chains, HRM maintains predictable consumption enabling reliable deployment without memory leaks. With only 27 million parameters, the HRM occupies approximately 108 MB in 32-bit floating-point representation, fitting comfortably within the memory constraints of edge devices such as Raspberry Pi 5 with 16 GB RAM [1].

However, the dual-module architecture introduces memory overhead compared to single-module recurrent networks. The high-level planning module and low-level worker module each maintain separate hidden state representations, doubling the base memory footprint relative to equivalent single-module architectures [1]. Inter-module communication at each high-level cycle, transferring converged low-level states and broadcasting updated high-level context, may introduce performance bottlenecks on ARM platforms with limited memory bandwidth.

3.2.5 Training Methodology: One-Step Gradient Approximation

Recurrent networks usually require BPTT for gradient computation, consuming $O(T)$ memory to store hidden states. HRM avoids this through one-step gradient approximation, achieving $O(1)$ memory complexity [1], building on Deep Equilibrium Model (DEQ) theory explored by Bai *et al.* [9]. In practice, HRM computes gradients only through the final states of each module, ignoring all other timesteps. The model also incorporates deep supervision, applying loss feedback at multiple stages during training through multiple forward passes, each producing a prediction. Figure 3.2 illustrates the hierarchical module architecture and one-step gradient approximation training.

However, recent analysis by Ge *et al.* [7] identifies a critical discrepancy: while Adaptive Computation Time (ACT) [10] learns halting policies during training, the original HRM evaluation always executes maximum reasoning steps, ignoring the learned policy.

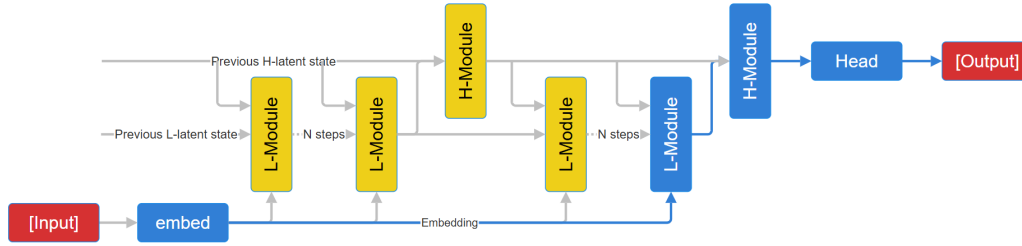


Figure 3.2: The HRM dual-module architecture as described by Wang *et al.* [1]. The input is embedded and fed to the L-module, which iterates for N steps per cycle. After each cycle, the H-module updates using the converged L-module state, resetting the L-module’s computational space for the next cycle. The output head produces the final prediction after all cycles complete. Modules coloured in blue experience one-step gradient approximation during training, greatly reducing memory requirements from $O(n)$ with BPTT to $O(1)$ with one-step gradient approximation.

When ACT is properly applied during inference, performance changes minimally, suggesting that the model’s strengths derive more from increased computational depth than from adaptive resource allocation.

Additionally, Ge *et al.* observe that one-step gradient approximation is functionally equivalent to training paradigms used in diffusion models [7]. This suggests HRM may be better understood as a deep feed-forward network rather than a genuinely recurrent architecture, challenging the originally proposed explanation for HRM’s effectiveness.

3.3 Performance and Benchmarks

The HRM’s empirical performance provides concrete evidence of its capabilities on definitive reasoning tasks.

3.3.1 Benchmark Tasks

The HRM was evaluated on three challenging reasoning benchmarks [1]. The Abstraction and Reasoning Corpus (ARC) [19] measures fluid intelligence through visual puzzles requiring inductive rule learning from 2–3 demonstration pairs. The benchmark includes ARC-AGI-1 (960 training examples) and ARC-AGI-2 (1,120 examples with more compositional reasoning).

Sudoku-Extreme comprises the most challenging Sudoku puzzles, requiring an average of 22 backtracking steps [12]. The dataset includes 1,000 training examples spanning easy (0 backtracks) to expert-level (50+ backtracks). Maze-Hard tests optimal pathfinding in 30×30 grids where the shortest path exceeds 110 steps. For all benchmarks, the HRM was trained from initialisation, using only 1,000 training examples.

3.3.2 Empirical Results

The HRM achieves considerable performance gains over state-of-the-art CoT models despite having orders of magnitude fewer parameters. On ARC-AGI-1, HRM attains 40.3% accuracy, surpassing o3-mini-high (34.5%), Claude 3.7 8K (21.2%), and DeepSeek R1

(21.0%) [1]. On ARC-AGI-2, HRM achieves 5.0% compared to 3.0% for o3-mini-high. These results are striking given that HRM operates with 27 million parameters while compared models have hundreds of billions.

On symbolic reasoning tasks demanding extensive search, the gap widens further. On Sudoku-Extreme, HRM achieves 55.0% accuracy while all CoT-based models as of August 2025 achieve 0% [1]. On Maze-Hard, HRM reaches 74.5% while tested CoT models (DeepSeek R1, ChatGPT 4.1, Claude Sonnet 3.7) failed entirely. For definitive reasoning tasks involving constraint satisfaction and systematic search, hierarchical iterative refinement offers qualitatively superior performance.

Table 3.1: Comparison of HRM and Chain-of-Thought LLMs across key dimensions.

Dimension	HRM	CoT LLMs
Parameters	27M	300B–2T
Training Data	1,000 examples	Tens of trillions of tokens
Sudoku-Extreme	55%	0%
ARC-AGI-1	40.3%	34.5% (best)
Maze-Hard	74.5%	0%
Memory Complexity	$O(1)$	$O(L)$

3.4 Comparative Analysis and Critical Perspectives

3.4.1 Advantages Over Chain-of-Thought Reasoning

The HRM offers distinct advantages for definitive reasoning tasks. Firstly, *data efficiency* dramatically reduces training requirements by three orders of magnitude compared to CoT-based approaches, which demand millions of annotated reasoning traces [20]. The model achieves strong performance with merely 1,000 input-output pairs per task, a result of learning in continuous latent space rather than linear token generation.

Second, *compact architecture* enables deployment on resource-constrained machines. The model’s 27 million parameters occupy approximately 108 MB of memory, compared to 700+ GB for GPT-4-scale models. The $O(1)$ memory complexity during training further reduces requirements, enabling single-GPU training and inference.

Third, *resistance to partial errors* separates HRM from CoT approaches. Because the model iteratively refines solutions holistically, errors in latent states can be corrected in later iterations. By contrast, CoT reasoning exhibits cascading failure: once an incorrect step is generated, it often propagates through the remainder of the chain [3, 4].

Lastly, HRM achieves a form of *Turing-completeness* when provided sufficient computational resources. Unlike fixed-depth Transformers constrained to AC^0/TC^0 complexity classes, the recurrent architecture can theoretically simulate Turing machines given adequate memory and time [1].

3.4.2 Limitations and Trade-offs

Despite these advantages, HRM has limitations. *Task scope* represents the most fundamental constraint: current implementations excel on two-dimensional grid-based reason-

ing but do not exhibit effectiveness on natural language tasks, open-ended generation, or domains requiring world knowledge.

Interpretability presents another challenge. While CoT reasoning produces human-readable steps, HRM operates entirely in latent space. Understanding why the model produces particular solutions requires analysing high-dimensional continuous trajectories, which is significantly more difficult than examining token sequences. For applications requiring explainable decision-making, this limitation is prohibitive.

Inference cost also merits consideration. Although HRM’s 27M parameters enable efficient deployment, solving complex problems may require hundreds of computational steps. For problems where CoT succeeds, token-by-token generation may prove more efficient than iterative latent refinement.

3.4.3 Recent Critical Findings

A recent study by Ge *et al.* [7] questions several assumptions in HRM’s design. Most significantly, ablation studies suggest the high-level module may provide minimal benefits: an HRM variant using only the L-module achieves comparable accuracy to the full system. This finding, confirmed by the ARC Foundation [8], raises fundamental questions about whether hierarchy is necessary or whether computational depth alone drives performance. This project investigates this question directly, as reported in Chapter 5.

These criticisms do not invalidate HRM’s empirical performance but suggest its success may derive from different mechanisms than originally proposed. The minimal role of the H-module and ineffectiveness of learned halting policies indicate that simple computational depth, achieved through any stable iterative refinement process, may suffice for definitive reasoning tasks. These findings motivate the L-module-only variant implemented in this project, as described in Chapter 4.

3.5 Biological Correspondence

The HRM demonstrates parallels with principles of brain function, suggesting plausible similarity to how brains reason.

Neuroscientific principles suggest that the function of a brain region is related to its structure [21, 22]. Higher-order areas responsible for complex reasoning exhibit higher activity, reflecting their capacity to represent task-relevant variables. This gives rise to a dimensionality hierarchy: effective dimensionality increases from low-level sensory areas to high-level associative cortex [23].

The HRM reproduces this organisation. Analysis using Participation Ratio (PR), a standard measure of effective dimensionality, reveals that the high-level module operates in higher-dimensional space ($PR = 89.95$) compared to the low-level module ($PR = 30.22$) when trained on Sudoku-Extreme [1]. This ratio closely matches the approximately 2.25:1 ratio observed between high-level and low-level areas in the cortex of a mouse [23]. Furthermore, the dimensionality of the H-module scales with the number of unique tasks, indicating an adaptive expansion of the representational capacity, while the L-module maintains a stable dimensionality. This mirrors biological observations that higher cortical areas show greater representational flexibility [21].

Importantly, this hierarchical organisation is an emergent property of training rather than an engineered feature. Analysis of an untrained HRM shows no separation of dimensionality: both modules have nearly identical PR values [1]. This suggests that the learned

separation between high-dimensional planning and low-dimensional execution arises from task demands rather than an imposed structure, paralleling Murray *et al.*'s findings that biological cortical hierarchies are organised across development [16].

3.6 Cross-Platform Inference Frameworks

Deploying HRMs across heterogeneous edge hardware requires intermediate representations that preserve model semantics while enabling efficient execution on diverse processor architectures. Cross-platform inference frameworks serve as the critical bridge between high-level model definitions produced during research and low-level execution on target edge devices.

3.6.1 ONNX Representation Challenges

The Open Neural Network Exchange (ONNX) format has emerged as the de facto standard for platform-agnostic model representation, defining a comprehensive operator set and computation graph structure that enables models trained in PyTorch or TensorFlow to execute across diverse inference engines without framework-specific dependencies [24, 25]. This portability proves particularly valuable for edge deployment scenarios where training environments on high-performance GPUs differ substantially from resource-constrained deployment platforms.

However, HRM characteristics present unique challenges distinguishing them from conventional feedforward networks. The iterative refinement process involves dynamic computation graphs where iteration counts vary based on problem complexity and input characteristics [18]. While ONNX supports recurrent operators through loop constructs, fixed-point iteration semantics require careful modelling to preserve convergence properties during the conversion process. Jiang *et al.* conducted an extensive analysis of ONNX converter failures, finding that approximately 75% of defects occur during node conversion and 33% of reported failures result in semantically incorrect models that produce different outputs than the original [26]. These findings raise significant concerns for HRM deployment, where convergence behaviour depends critically on precise numerical semantics being preserved throughout the conversion pipeline.

The dual-module architecture of HRMs compounds these conversion challenges considerably. Unlike autoregressive models processing inputs through single feedforward passes, HRMs require coordinated execution of high-level planning and low-level worker modules operating at different temporal scales [1]. Representing this hierarchical coupling in ONNX necessitates explicit modelling of inter-module communication patterns, including the reset signals that enable hierarchical convergence.

3.6.2 Runtime Optimisation Applicability

ONNX Runtime provides a comprehensive inference engine employing optimisation techniques including operator fusion, constant folding, and memory layout transformations that substantially improve inference throughput for conventional architectures [27, 28]. However, these optimisations were designed primarily for CNNs and Transformers rather than iterative reasoning models, raising fundamental questions about their effectiveness for HRM deployment.

Operator fusion opportunities that benefit feedforward networks may not generalise effectively to recurrent architectures where state dependencies between iterations constrain operator reordering possibilities [29]. The runtime’s memory management strategies assume relatively static consumption patterns, whereas HRMs exhibit variable memory requirements correlated with iteration counts on individual problem instances.

ARM NEON SIMD extensions on Cortex-A76 processors (Raspberry Pi 5) offer vectorised operations achieving 2–4× speedups for neural inference, representing a critical optimisation opportunity for edge deployment. However, ONNX Runtime’s ARM-specific optimisations primarily target the inference patterns observed in classification and object detection networks [24]. The extent to which these optimisations benefit HRM’s iterative refinement patterns remains an open empirical question, and the absence of HRM-specific deployment guidelines represents a significant gap in current tooling [25].

3.7 Edge Device Optimisation Techniques

Deploying HRMs on resource-constrained edge devices requires optimisation strategies that specifically account for the unique computational characteristics of iterative reasoning architectures. While conventional neural network optimisation techniques provide a starting point, the fixed-point iteration dynamics and hierarchical convergence mechanisms inherent to HRMs introduce challenges absent in feedforward or autoregressive models.

3.7.1 Quantisation for Iterative Architectures

Quantisation reduces numerical precision from 32-bit floating-point to lower bit-widths such as 16-bit half-precision or 8-bit integers, decreasing memory footprint and potentially accelerating inference on hardware with specialised integer arithmetic units [30, 31]. For edge deployment on ARM platforms with limited floating-point throughput, quantisation often proves essential for achieving acceptable inference latency within power budgets. However, the interaction between reduced numerical precision and fixed-point iteration convergence introduces complications entirely absent in single-pass feedforward networks.

Quantisation introduces rounding errors at each computational step, and in iterative architectures these errors accumulate across successive iterations in potentially complex ways. For HRMs performing dozens to hundreds of refinement cycles, accumulated quantisation error may destabilise convergence, cause oscillatory behaviour, or substantially increase the iteration count required to reach acceptable solution quality [32]. Theoretical analysis of quantised fixed-point iteration suggests that convergence guarantees depend critically on maintaining sufficient numerical precision to preserve the contraction mapping properties of the recurrent operator [9]. If quantisation-induced perturbations exceed convergence tolerances determined by the operator’s spectral properties, the system may fail to converge entirely or oscillate indefinitely between approximate solutions.

Mixed-precision quantisation strategies offer a potential compromise by allocating higher precision to convergence-critical operations while applying aggressive quantisation to less sensitive components [32]. For HRMs specifically, the high-level planning module’s operations may tolerate lower precision given its slower update frequency, while the low-level worker module’s rapid iterative refinement may require higher precision to maintain convergence stability. However, identifying the optimal precision allocation requires extensive empirical evaluation on target hardware and across task distributions,

as the relationship between precision and convergence behaviour proves highly problem-dependent. Quantisation-aware training approaches that simulate quantisation effects during model training may help develop networks inherently robust to reduced precision [31], though adapting these techniques to hierarchical iterative architectures represents an unexplored research direction.

3.7.2 ARM-Specific Optimisations

ARM processors dominate the edge computing landscape, with platforms such as Raspberry Pi 5 employing ARM Cortex-A76 cores featuring NEON SIMD instruction sets capable of performing vectorised operations on multiple data elements simultaneously, offering 2–4 $\times$ speedups for conventional neural network inference workloads [33]. Effective exploitation of these capabilities substantially improves inference throughput, but vectorisation for recurrent architectures proves fundamentally more challenging than for feedforward networks due to the data dependencies between successive iterations that constrain parallelisation opportunities.

HRM computation introduces memory access patterns that may poorly match ARM cache hierarchies optimised for sequential or strided access patterns. At each iteration, the recurrent operator must load hidden states from memory, perform matrix computations, and write updated states back to memory. On ARM platforms with limited cache capacity and moderate memory bandwidth, these repeated memory transfers may constitute performance bottlenecks that dominate computational costs [34]. Cache-aware scheduling strategies that maximise temporal locality, keeping frequently accessed hidden states resident in cache across iterations, prove essential for minimising memory traffic and improving effective bandwidth utilisation.

The dual-module architecture of HRMs further complicates parallelisation strategies. The high-level and low-level modules operate at different temporal scales, with the low-level module executing multiple iterations per high-level planning cycle [1]. On multi-core ARM processors, distributing modules across cores could theoretically enable parallel execution, but the frequent inter-module communication required for hierarchical convergence may introduce synchronisation overhead that negates parallelisation benefits. Recent benchmarking studies demonstrate that memory bandwidth limitations often constrain achievable parallelism for neural network workloads on edge platforms [33], suggesting that careful empirical evaluation of parallelisation trade-offs proves necessary for HRM deployment.

3.8 Research Gaps and Summary

Table 3.2 summarises the critical research gaps identified across the domains examined in this review. The absence of empirical studies addressing the intersection of HRM architectures with edge deployment requirements represents the most significant limitation in current literature, leaving developers without guidance for this promising deployment scenario.

This project addresses several of these gaps directly. The L-module-only variant tests whether hierarchical structure is necessary, the Weighted Maze task extends evaluation to cost-sensitive pathfinding, and inference benchmarking on Raspberry Pi 5 provides the first empirical data on HRM performance on ARM-based edge hardware.

Table 3.2: Critical research gaps in HRM edge deployment.

Domain	Research Gap
HRM Architecture	No ARM hardware benchmarks for iterative reasoning architectures; unclear whether hierarchical structure is necessary
Cross-Platform	ONNX converter issues with dynamic control flow; no HRM-specific deployment guidelines
Edge Optimisation	Unknown quantisation effects on fixed-point convergence stability
Novel Tasks	HRM evaluated only on unweighted grid tasks; no cost-sensitive pathfinding benchmarks
Integration	No comprehensive Raspberry Pi or similar edge deployment studies for HRM

Chapter 4

System Design

This chapter provides a detailed description of the system architecture and design decisions made throughout the project. It covers the overall structure of the codebase, the architecture of both model variants (full HRM and L-module-only), the encoding and generation of both reasoning tasks (Sudoku and Weighted Maze), and the training and evaluation pipelines. Where relevant, UML diagrams, architecture figures, and code snippets are used to illustrate the design. The ONNX export pipeline, the continuous integration infrastructure, and the Raspberry Pi 5 deployment are described in Kyrylo Kozlovskyi’s dissertation, where they are presented in greater depth. References to those components are included here where they intersect with the material I led.

4.1 Overall System Architecture

The project is structured as a Python package hosted at <https://github.com/fionntmcc/cross-platform-hrm>. The repository follows a modular design with clear separation between model definitions, dataset generation, training logic, and evaluation. Table 4.1 summarises the top-level directory structure.

Table 4.1: Repository directory structure.

Directory / File	Purpose
hrm/	Core package containing model definitions, dataset generators, training scripts, and utility functions
scripts/	Entry points for training, inference, dataset generation, ONNX export, and visualisation
tests/	Unit and integration tests for model components and data pipelines
docs/	Project documentation
.github/workflows/	GitHub Actions CI/CD pipeline for automated testing
requirements.txt	Python dependencies

The implementation is built entirely in Python using PyTorch as the deep learning framework. A deliberate design decision was made to avoid CUDA-specific extensions such as FlashAttention, which the original Sapiient Inc. codebase requires [1]. This choice

sacrifices some training speed on high-end GPUs but ensures that the codebase runs on any platform with a standard PyTorch installation, including CPU-only environments and ARM-based edge devices such as the Raspberry Pi 5. This cross-platform compatibility is reflected in the repository name and represents a key differentiator from the original implementation.

Code quality is enforced through a continuous integration pipeline implemented by Kyrylo Kozlovskiy using GitHub Actions, which runs Black formatting, Ruff linting, and the pytest test suite on every push.

Figure 4.1 illustrates the high-level data flow through the system, from dataset generation through training to evaluation on multiple hardware targets.

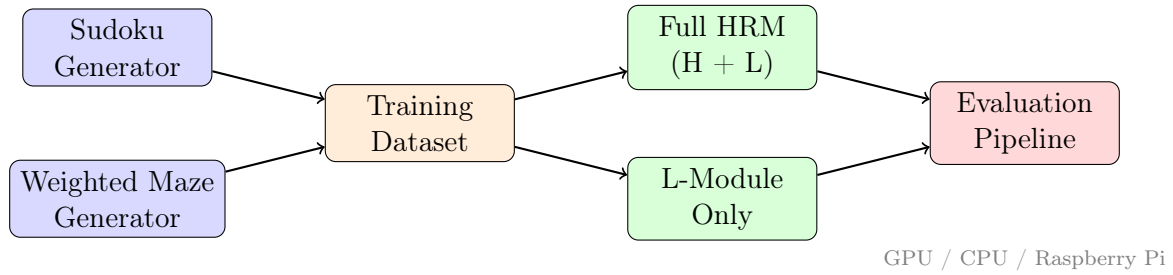


Figure 4.1: High-level system architecture showing the pipeline from dataset generation through model training to evaluation across multiple hardware targets.

4.2 Model Architecture: Full HRM

The prototype full HRM implementation, which I led the development of, follows the architecture described by Wang *et al.* [1] and is based on the official Sapien Inc. codebase¹. The model consists of four networks: an input network, a low-level recurrent module (L-module), a high-level recurrent module (H-module), and an output network. The full HRM uses $N = 16$ high-level cycles of $T = 16$ low-level steps each, yielding 256 total computational steps per forward pass.

4.2.1 Input Network

The input network $f_I(\cdot; \theta_I)$ converts the raw puzzle representation into a dense embedding suitable for processing by the recurrent modules. For both Sudoku and Weighted Maze tasks, the input is a two-dimensional grid of integer tokens. Each token value is mapped to a learned embedding vector, producing a three-dimensional tensor of shape (height $\times$ width $\times$ embedding dimension) that serves as the initial representation for the recurrent modules.

4.2.2 Recurrent Modules

The L-module $f_L(\cdot; \theta_L)$ is the core computational engine of the model. At each timestep t within a high-level cycle, it updates its hidden state according to:

$$h_L^{(t)} = f_L(h_L^{(t-1)}, h_H, x) \quad (4.1)$$

¹<https://github.com/sapieninc/HRM/>

where $h_L^{(t-1)}$ is the previous L-module hidden state, h_H is the current H-module state (held constant within a cycle), and x is the input representation. The L-module performs $T = 16$ iterations per cycle, converging toward a local fixed point determined by the current H-module context.

The H-module $f_H(\cdot; \theta_H)$ operates at a slower timescale. It updates once per cycle using the converged L-module state:

$$h_H^{(n)} = f_H(h_H^{(n-1)}, h_L^{(T)}) \quad (4.2)$$

where $h_L^{(T)}$ is the final L-module state from cycle n . This update effectively resets the L-module’s computational space, directing it toward a new equilibrium in the next cycle. The total computation unfolds over $N = 16$ high-level cycles of $T = 16$ low-level steps each, yielding 256 total computational steps per forward pass.

Both the L-module and the H-module were implemented as transformer blocks using multi-head self-attention with rotary positional embeddings (RoPE) and SwiGLU-activated MLPs. The model’s compact size of approximately 27 million parameters arises not from any absence of attention, but from two architectural choices: a relatively small hidden dimension (256), and aggressive weight sharing across iterative reasoning steps. Each transformer block is applied repeatedly with the same parameters rather than being stacked into a deeper network with distinct weights at each layer, which keeps the parameter budget low while still delivering large effective computational depth.

4.2.3 Output Network

The output network $f_O(\cdot; \theta_O)$ maps the final L-module hidden state to a prediction over the output space. For Sudoku, this produces a probability distribution over digits 1–9 for each cell. For Weighted Maze, it produces a binary classification (on or off the optimal path) for each cell. During training, the output network is applied at the end of each high-level cycle as part of the deep supervision strategy, providing a gradient signal at multiple points along the iterative refinement trajectory rather than only at the final step.

4.2.4 Training with One-Step Gradient Approximation

The model is trained using the one-step gradient approximation described in Section 3.2. Rather than backpropagating through all 256 timesteps (which would require $O(N \times T)$ memory), gradients are computed only through the final state of each module, achieving $O(1)$ memory complexity. This is critical for training on a single consumer GPU (NVIDIA RTX 5070 Ti Laptop GPU with 12 GB GDDR7), as full BPTT would be infeasible given the hundreds of computational steps involved.

Deep supervision is applied by computing the loss at the output of multiple intermediate cycles, not just the final one. Each forward pass through the training loop produces predictions at the end of several cycles, and the total loss is the sum of these intermediate losses. This provides a richer gradient signal and helps stabilise training, particularly in the early stages when the model has not yet learned effective iterative refinement patterns.

4.3 Model Architecture: L-Module Only

The L-module-only variant tests the hypothesis, discussed in Chapter 3, that hierarchical structure may be unnecessary for effective reasoning. This variant removes the H-module

entirely while keeping the input network, L-module, and output network unchanged. With the H-module removed, the parameter count drops from approximately 27 million to approximately 6.9 million, while preserving the same hidden dimension (256), the same SwiGLU-activated transformer block structure, the same rotary positional embeddings, and the same weight-sharing scheme as the full HRM. The variant was implemented and led by Kyrylo Kozlovskyi, who reports its precise configuration in his dissertation.

Without the H-module, the L-module no longer receives periodic resets that redirect its computation toward new equilibria. Instead, the L-module simply iterates for the full $N \times T$ steps in a single continuous sequence:

$$h_L^{(t)} = f_L(h_L^{(t-1)}, x) \quad \text{for } t = 1, 2, \dots, N \times T \quad (4.3)$$

Note the absence of h_H in the update rule. The H-module state is no longer provided as conditioning input, and no hierarchical reset occurs between cycles. The total number of computational steps remains the same, isolating the effect of the hierarchical structure from the effect of computational depth.

This design follows the ablation methodology used by Ge *et al.* [7]. The reduced parameter count also decreases memory consumption, which is relevant for edge deployment.

The training procedure for the L-module-only variant is identical to the full HRM: one-step gradient approximation with deep supervision and the same hyperparameters. I implemented the one-step backpropagation mechanism for this variant, adapting the gradient approximation from the full HRM to work without the H-module’s periodic resets. This ensures that any differences in performance can be attributed to the architectural change rather than to training configuration.

4.4 Task Implementation: Sudoku

The Sudoku task follows the protocol established by Wang *et al.* [1]. Puzzles are represented as 9×9 grids where each cell contains an integer token.

4.4.1 Input Encoding

The input encoding uses a simple integer scheme:

- **0**: empty cell (to be solved)
- **1–9**: pre-filled digit values (given clues)

Each token is mapped to a learned embedding vector by the input network before being passed to the recurrent modules. The model receives the partially filled grid and must produce the complete solution.

4.4.2 Output Representation

The output network produces a probability distribution over digits 1–9 for each of the 81 cells. During training, the loss is computed using cross-entropy between the predicted distribution and the ground-truth solution for all cells, including both given clues and empty cells. Including the clue cells in the loss provides additional supervision signal and

trains the model to learn that the given values must be preserved as part of the solution, rather than treating them as fixed constants outside the model’s responsibility. During evaluation, the predicted digit for each cell is taken as the argmax of the output distribution. No constraint enforcement is applied at inference time, so the model’s prediction for a pre-filled cell is reported as-is rather than being constrained to the original given value, and a puzzle is considered solved only if every cell (including all pre-filled cells) matches the ground truth exactly.

4.4.3 Dataset Generation

I developed a custom Sudoku puzzle generator for this project rather than relying on the external Sudoku-Extreme dataset used in the original HRM implementation [12]. The generator produces puzzles from scratch: it first constructs a valid complete grid using a backtracking solver with randomised number selection, then progressively removes clues while verifying at each step that the puzzle retains exactly one unique solution.

Difficulty is controlled through the number of backtracks required to solve the resulting puzzle, matching the difficulty metric used in the Sudoku-Extreme dataset [12]. Three difficulty levels are supported: easy (0–4 backtracks), medium (5–15 backtracks), and hard (15–100 backtracks). A mixed mode generates equal proportions of each difficulty level. The generator also supports 4×4 grids for rapid development and debugging, where difficulty is instead measured by the number of empty cells.

The generator outputs problems and solutions as NumPy arrays of shape (number of puzzles × grid size × grid size), stored in compressed `.npz` format alongside a metadata file recording the difficulty level, backtrack count, and number of empty cells for each puzzle. A deduplication step ensures that all puzzles in a dataset are unique. The generator is fully configurable via command-line arguments, supporting custom output paths, random seeds for reproducibility, and batch sizes.

For training, the model uses 10,000 puzzles generated in mixed difficulty mode, matching the difficulty distribution described by Wang *et al.* [1]. A held-out test set is reserved for evaluation.

4.5 Task Implementation: Weighted Maze

The Weighted Maze task was designed and implemented by me. Unlike the Maze-Hard task in the original HRM evaluation [1], which uses uniform movement costs and evaluates shortest path by step count, the Weighted Maze assigns traversal costs to cells, requiring the model to reason about cumulative cost to find the cheapest path. The standard configuration used for training and evaluation throughout this project is a 15×15 grid, although the generator supports arbitrary grid sizes for development and qualitative analysis.

4.5.1 Input Encoding

The input is a two-dimensional grid where each cell contains an integer token with the following semantics:

This encoding, which I designed to keep the input format consistent with the Sudoku task, allows the model to distinguish between walls, the start and goal positions, standard-cost cells, and cells with elevated traversal costs ranging from 4 to 9, all within

Table 4.2: Weighted Maze input token encoding.

Token	Meaning
0	Wall
1	Open path (movement cost = 1)
2	Start position
3	Goal position
4–9	Weighted path cell (movement cost = token value)

a single integer. Identically to the Sudoku task, the token values are mapped to learned embeddings by the input network.

4.5.2 Output Representation

The output for each cell is a binary classification:

- **0**: not on the optimal (minimum-cost) path
- **1**: on the optimal path

The optimal path is computed during dataset generation using Dijkstra’s algorithm, which finds the path from start to goal that minimises the total traversal cost. To guarantee a unique ground truth, the generator detects cases in which multiple distinct paths share the same minimum cost and changes individual cell weights by small amounts until exactly one path is optimal. This avoids ambiguity in the supervision signal, since training the model against a non-unique ground truth would penalise it for producing valid solutions that differ from the arbitrarily selected reference path. During evaluation, a maze is considered solved correctly only if the model’s predicted path exactly matches the optimal path.

4.5.3 Dataset Generation

I developed a custom Weighted Maze generator for this project. The generator produces grids of configurable size with randomly placed walls, a start position, a goal position, and weighted cells. The generation process ensures that at least one valid path exists from start to goal, and that the optimal path is made unique through the perturbation step described above. The optimal path is computed using Dijkstra’s algorithm over the grid graph, where edge weights correspond to the destination cell’s token value.

The key difference between this task and the original Maze-Hard benchmark is that the model cannot simply find the shortest path by step count. Two paths of equal length may have very different total costs, forcing the model to learn a notion of cost accumulation within its latent space. The model must weigh trade-offs between shorter paths through expensive cells and longer paths through cheaper cells, introducing a fundamentally different reasoning challenge.

4.6 Training Pipeline

The training pipeline is shared between both model variants and both tasks. Figure 4.2 illustrates the training loop.

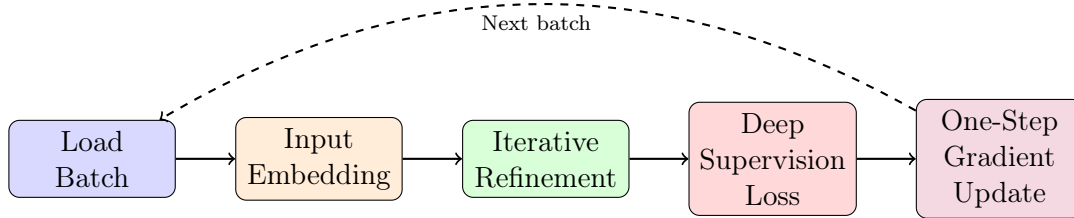


Figure 4.2: Training pipeline showing the flow from data loading through iterative refinement to gradient update using one-step gradient approximation with deep supervision.

The training occurs as follows. For each batch, the input grid is passed through the input network to produce an embedding. The recurrent modules then perform $N \times T$ iterations of refinement (with or without the H-module depending on the variant). At several intermediate checkpoints during this iteration, the output network produces a prediction and the loss is computed against the ground truth. The total loss is the sum of these intermediate losses. Gradients are computed using the one-step approximation and the model parameters are updated using the Adam optimiser.

All training was conducted on a single NVIDIA GeForce RTX 5070 Ti GPU with 12 GB GDDR7. Hyperparameters were kept identical between the full HRM and L-module-only variant for each task to ensure a fair comparison. Specific hyperparameter values (learning rate, weight decay, batch size, number of epochs, N , T) are reported alongside results in Chapter 5.

I implemented a training performance tracking system to monitor convergence behaviour throughout training. At each epoch, the system logs training loss, cell accuracy, puzzle accuracy, and validation loss to CSV files, providing a structured record of the model’s progress over time. The system also generates matplotlib plots automatically at the end of each training run, visualising these metrics as curves over epochs. This infrastructure proved valuable for diagnosing training issues, comparing runs with different configurations, and producing the results presented in Chapter 5.

4.7 Evaluation Pipeline

The evaluation pipeline measures accuracy and inference latency across three hardware configurations: GPU (NVIDIA RTX 5070 Ti), CPU (Intel i7-12700H laptop without GPU acceleration), and edge device (Raspberry Pi 5 with 16 GB RAM running Debian 13 Trixie, 64-bit).

For each hardware target, the trained model checkpoint is loaded and run in inference mode (no gradient computation) on the held-out test set. Accuracy is computed as described in the respective task sections above. Inference latency is measured as the wall-clock time per puzzle, averaged over the full test set to account for variance.

On the GPU and CPU targets, inference runs through PyTorch directly using the trained .pt checkpoint. On the Raspberry Pi 5, the model is exported to ONNX format and run via ONNX Runtime with no PyTorch dependency. The ONNX export pipeline

and zero-dependency inference script are described in Kyrylo Kozlovskyi’s dissertation. No quantisation is applied, so the model runs in standard 32-bit floating-point precision throughout, providing a clean baseline for edge hardware performance.

4.8 Trajectory Visualiser

I implemented a trajectory visualiser that displays the model’s intermediate predictions at each reasoning step during inference. For a given puzzle, the visualiser renders the model’s output at each of the $N \times T$ iterative steps as a colour-coded grid, alongside per-step metrics including cell accuracy, path accuracy (for mazes), and the number of cells that changed since the previous step (Δ). Figure 4.3 shows an example output for a 15×15 weighted maze.



Figure 4.3: Trajectory visualiser output for a 15×15 weighted maze, showing the model’s prediction at each of 16 reasoning steps as a colour-coded grid, allowing for iterative refinement behaviour to be inspected directly.

The visualiser also produces side-by-side comparison figures showing the model’s final prediction against the ground truth, with a colour legend distinguishing correct path cells, false positives, missed path segments, walls, floor cells, weighted cells, and the start and goal positions. These comparison figures are used in Chapter 5 to analyse the model’s behaviour on both correctly and incorrectly solved mazes.

This tool proved valuable throughout development for diagnosing convergence issues, understanding how the model allocates its reasoning steps, and verifying that iterative refinement was functioning as intended. It also provides a compelling visual demonstration of the core HRM concept: progressive solution refinement through latent reasoning and latent discovery of complex algorithms in latent space.

4.9 Design Decisions and Trade-offs

Several deliberate design decisions were made throughout the project, each involving trade-offs that are worth documenting.

Cross-platform compatibility over training speed. As discussed in Section 4.1, avoiding CUDA-specific extensions ensures portability at the cost of training speed.

Identical hyperparameters between variants. Using the same training configuration for both the full HRM and L-module-only variant simplifies the experimental comparison but may not produce the optimal result for each variant individually. The L-module-only variant, having fewer parameters, might benefit from a different learning rate or regularisation strategy. This is acknowledged as a limitation and discussed further in Chapter 5; tuning each variant independently is identified as future work.

Simple token encoding for Weighted Maze. The encoding scheme uses integer tokens 0–9 within a single channel, rather than a multi-channel representation that might separate wall, path, and cost information. This keeps the input format consistent with the Sudoku task and avoids modifying the input network architecture, but places a greater burden on the embedding layer to disentangle the different semantic meanings of token values.

Dijkstra’s algorithm for ground truth. Using Dijkstra’s algorithm guarantees optimal paths in the training data, providing clean supervision. However, this means the model is trained to replicate an algorithm that is itself $O(V \log V)$ in complexity. Whether HRM can learn to approximate this algorithm through iterative latent refinement is an open question that the evaluation chapter addresses.

Chapter 5

System Evaluation

This chapter evaluates the project against the objectives set out in Chapter 1 and presents the results of training and benchmarking experiments. Results are reported for the L-module-only variant across both the Sudoku and Weighted Maze tasks. The full HRM has been implemented on a separate branch, but benchmarking focuses on the L-module-only variant for the reasons discussed in Section 5.7.

5.1 Experimental Setup

All training experiments were conducted on a single NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12 GB GDDR7. Mixed-precision training (AMP) was enabled throughout. Inference was benchmarked on three platforms: GPU (the same RTX 5070 Ti), CPU (an Intel i7-12700H laptop without GPU acceleration), and a Raspberry Pi 5 with 16 GB RAM running Debian 13 Trixie (64-bit). Raspberry Pi 5 inference uses ONNX Runtime with no PyTorch dependency.

For both tasks, the L-module-only variant was trained from random initialisation using one-step gradient approximation with deep supervision, as described in Chapter 4. The final accuracy figures were computed by averaging the results in 10 independent training runs, each using a different random seed and evaluating 1,000 test puzzles to account for the variance in the training outcomes. Kyrylo Kozlovskyi’s dissertation reports separate ONNX Runtime benchmarks using a single seed with 500 puzzles per task, which yield slightly different accuracy figures due to the different evaluation protocol; the results reported here use the multi-seed averaged figures. Hyperparameter values are reported in Table 5.1.

5.2 Sudoku Results

The L-module-only variant was trained on a custom-generated dataset of mixed-difficulty 9×9 Sudoku puzzles. The difficulty distribution was designed to closely match the Sudoku-Extreme dataset used in the original HRM paper [1, 12], spanning the full range from easy puzzles (fewer than 5 backtracks required to solve) through to expert-level puzzles requiring 50 or more backtracks. The custom generator described in Chapter 4 produces this distribution from scratch rather than relying on the external Sudoku-Extreme corpus, which allows the difficulty distribution to be reproduced and adjusted as needed. Table 5.2 presents the evaluation results averaged across 10 seeds.

Table 5.1: Training hyperparameters used for both tasks.

Hyperparameter	Sudoku	Weighted Maze
Learning Rate	3×10^{-4}	3×10^{-4}
Weight Decay	0.1	0.1
Batch Size	512	128
Epochs	200	200
Reasoning Steps	16	16
Grid Size	9×9	15×15
Training Examples	1,000	1,000
LR Schedule	Cosine Annealing	Cosine Annealing
Mixed Precision (AMP)	Enabled	Enabled
Evaluation Seeds	10	10
Test Puzzles per Seed	1,000	1,000

Table 5.2: L-module-only performance on mixed-difficulty 9×9 Sudoku, averaged across 10 seeds.

Metric	Result
Puzzle Accuracy	87.82%
Token Accuracy	96.2%

The model achieves 87.82% puzzle accuracy averaged across 10 seeds with 1,000 puzzles each. The token accuracy reaches 96.2%, indicating that even in unsolved puzzles, the vast majority of individual cells are predicted correctly. Figure 5.1 shows the training curves over the course of training, and Figure 5.2 shows the evaluation script running on the test set with example predictions.

5.2.1 Analysis

The 87.82% puzzle accuracy demonstrates that the L-module-only variant learns effective Sudoku-solving behaviour from limited training data. For context, the original HRM paper reports 55% accuracy on Sudoku-Extreme using the full 27M-parameter two-module architecture [1]. Direct comparison requires care: the original evaluation focuses on the hardest end of the distribution (averaging 22 backtracks per puzzle), whereas the dataset used here spans the full difficulty range from easy to expert. The result is best read as evidence that the simplified architecture retains meaningful Sudoku-solving capability across a representative difficulty distribution, consistent with Ge *et al.* [7].

The training curves in Figure 5.1 reveal several notable patterns. The loss plot (top left) shows training loss decreasing steadily while validation loss plateaus around 0.35, indicating some overfitting in later epochs. Token accuracy (top right) rises rapidly in the first 40 epochs before plateauing near 96%, while puzzle accuracy (bottom left) continues to climb more gradually, reflecting the strict all-or-nothing evaluation criterion. The convergence residual plot (bottom right) shows the average L2 norm of hidden state changes across reasoning steps. The increasing residual over training suggests that the

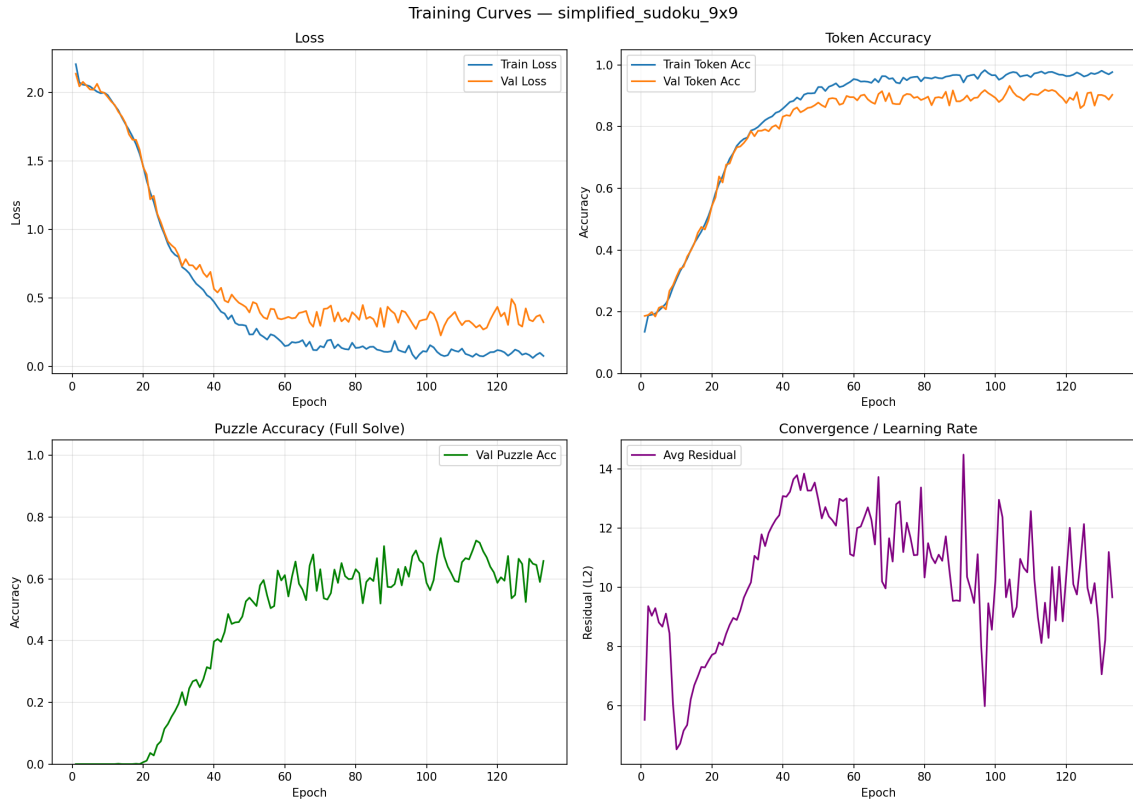


Figure 5.1: Sudoku 9×9 training curves. Top left: training and validation loss decreasing over epochs. Top right: token accuracy rising to approximately 96% on the validation set. Bottom left: puzzle accuracy climbing to approximately 70% on the validation set during this particular training run. Bottom right: average L2 residual across reasoning steps, showing the model’s internal convergence behaviour.

model learns to make larger, more deliberate updates at each reasoning step rather than collapsing to a near-static fixed point too early in the iteration sequence, which would otherwise indicate that the additional reasoning steps were doing no useful work.

The gap between token accuracy (96.2%) and puzzle accuracy (87.82%) reflects the strict evaluation criterion: a single incorrect cell out of 81 is sufficient to count the entire puzzle as failed. On average, unsolved puzzles contain only three to four incorrect cells, indicating that the model is close to a correct solution even when it fails.

5.3 Weighted Maze Results

The L-module-only variant was trained on 15×15 weighted mazes generated using the custom dataset generator described in Chapter 4. Table 5.3 presents the evaluation results averaged across 10 seeds.

The model achieves 75.20% puzzle accuracy averaged across 10 seeds, meaning that approximately 752 out of 1,000 test mazes had their optimal minimum-cost path predicted exactly. Cell accuracy exceeds 98%, indicating that even on incorrectly solved mazes, the model’s predictions are very close to the optimal solution. Figure 5.3 shows the training curves, and Figure 5.4 shows the evaluation script running on the test set.

```
(venv) PS C:\dev-stuff\final-year-project\cross-platform-hrm> python scripts/run_simplified.py --model model/simplified_hrm_sudoku_9x9_best.pt --puzzle sudoku_9x9 --data data/sudoku_9x9_test.json --eval-count 1000

Loaded 1000 puzzles from data\sudoku_9x9_test.json

Evaluating on 1000 puzzles ...

=====
EVALUATION RESULTS (1000 puzzles)
=====
Token accuracy (empty cells) : 96.2%
Puzzle accuracy (all correct): 879/1000 (87.9%)
Valid Sudoku solutions      : 867/1000 (86.7%)
=====

=====
EXAMPLE SOLUTIONS
Legend: bold=given  green=correct  red=wrong
=====

-- Puzzle 1 (valid) -- 24/24 empty cells correct --

Model prediction:


|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 3 | 4 | 8 | 2 | 5 | 9 | 1 | 7 | 6 |
| 2 | 5 | 9 | 1 | 6 | 7 | 8 | 4 | 3 |
| 1 | 7 | 6 | 3 | 4 | 8 | 9 | 5 | 2 |
| 5 | 3 | 4 | 8 | 7 | 6 | 2 | 1 | 9 |
| 9 | 8 | 7 | 5 | 2 | 1 | 3 | 6 | 4 |
| 6 | 1 | 2 | 9 | 3 | 4 | 5 | 8 | 7 |
| 4 | 9 | 3 | 6 | 1 | 5 | 7 | 2 | 8 |
| 7 | 2 | 1 | 4 | 8 | 3 | 6 | 9 | 5 |
| 8 | 6 | 5 | 7 | 9 | 2 | 4 | 3 | 1 |



Ground truth:


|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 3 | 4 | 8 | 2 | 5 | 9 | 1 | 7 | 6 |
| 2 | 5 | 9 | 1 | 6 | 7 | 8 | 4 | 3 |
| 1 | 7 | 6 | 3 | 4 | 8 | 9 | 5 | 2 |
| 5 | 3 | 4 | 8 | 7 | 6 | 2 | 1 | 9 |
| 9 | 8 | 7 | 5 | 2 | 1 | 3 | 6 | 4 |
| 6 | 1 | 2 | 9 | 3 | 4 | 5 | 8 | 7 |
| 4 | 9 | 3 | 6 | 1 | 5 | 7 | 2 | 8 |
| 7 | 2 | 1 | 4 | 8 | 3 | 6 | 9 | 5 |
| 8 | 6 | 5 | 7 | 9 | 2 | 4 | 3 | 1 |


```

Figure 5.2: Evaluation script output for a single training run on 1,000 test puzzles, yielding 96.2% token accuracy and 87.9% puzzle accuracy. An example solved puzzle is displayed with given clues in bold, correct predictions in green, and incorrect predictions in red.

Table 5.3: L-module-only performance on 15×15 Weighted Maze, averaged across 10 seeds.

Metric	Result
Puzzle Accuracy	75.20%
Cell Accuracy	>98%

5.3.1 Prediction Visualisation

To better understand the model’s behaviour, Figures 5.5 and 5.6 show side-by-side comparisons of the model’s prediction against the ground truth for a correctly solved and an incorrectly solved maze respectively.

The comparison between Figures 5.5 and 5.6 illustrates the characteristic failure mode discussed above: even on incorrectly solved mazes, the model’s prediction closely resembles the ground truth, with errors concentrated at specific decision points where alternative routes have similar total costs.

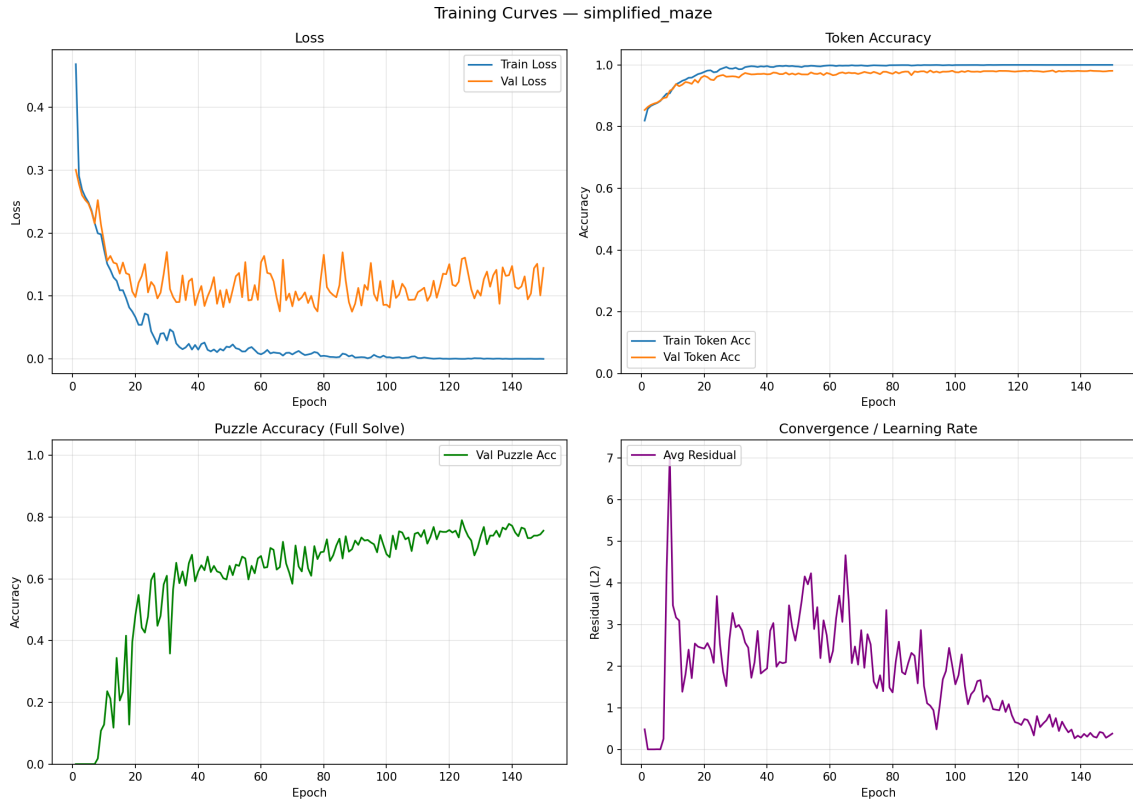


Figure 5.3: Weighted Maze 15×15 training curves. Top left: training loss. Top right: token accuracy. Bottom left: puzzle accuracy. Bottom right: average L2 residual.

5.3.2 Trajectory Visualisation

Figure 5.7 shows the output of the trajectory visualiser, which I implemented to inspect the model’s iterative refinement process. The figure displays the model’s prediction at each of the 16 reasoning steps for a single maze, along with cell accuracy, path accuracy, and the number of changed cells (Δ) at each step.

The trajectory visualisation reveals several important characteristics of the iterative reasoning process. The model’s initial prediction (Step 1) is a rough approximation that captures the general direction of the path but contains significant errors, particularly false positives (red cells) and missed segments (purple cells). Over the next three to four steps, the model rapidly refines this approximation, eliminating false positives and filling in missed segments. By Step 5, the prediction has converged to the correct solution with no further changes in subsequent steps. This convergence pattern suggests that the model uses the early reasoning steps for coarse spatial reasoning and the later steps for fine-grained correction, consistent with the iterative refinement paradigm described in the HRM literature [1].

The fact that convergence occurs by Step 5 out of 16 total steps also suggests that the configured step count provides substantial headroom for this particular maze. More complex mazes (larger grids, more ambiguous cost trade-offs) may require additional steps to converge, which could be investigated by varying the step count and measuring the effect on accuracy.


```
(venv) PS C:\dev-stuff\final-year-project\cross-platform-hrm> python scripts/run_simplified.py --model model/simplified_hrm_maze_best.pt --puzzle maze --data
data/maze_15x15_test.json --eval-count 1000
=====
Cell accuracy : 98.0%
Puzzle accuracy : 764/1000 (76.4%)
=====

EXAMPLE SOLUTIONS
Legend: green=correct path red=wrong path cyan=missed
=====

-- Maze 1 -- 225/225 cells correct --

Model prediction:
I
6 6 5 5 9 9 5 7
8 9 5 6
9 9 8 7
9 4 5 8 8 8 5
8 9 7 4 5 6
9 7 8 7 5 8 8
6 8 7 9 5 7

Ground truth:
6 6 5 5 9 9 5 7
8 9 5 6
9 9 8 7
9 4 5 8 8 8 5
8 9 7 4 5 6
9 7 8 7 5 8 8
6 8 7 9 5 7
```

Figure 5.4: Evaluation script output for a single test run on 1,000 test mazes, showing 98.0% cell accuracy and 76.4% puzzle accuracy. Below the summary, an example of the model’s correct prediction is displayed.

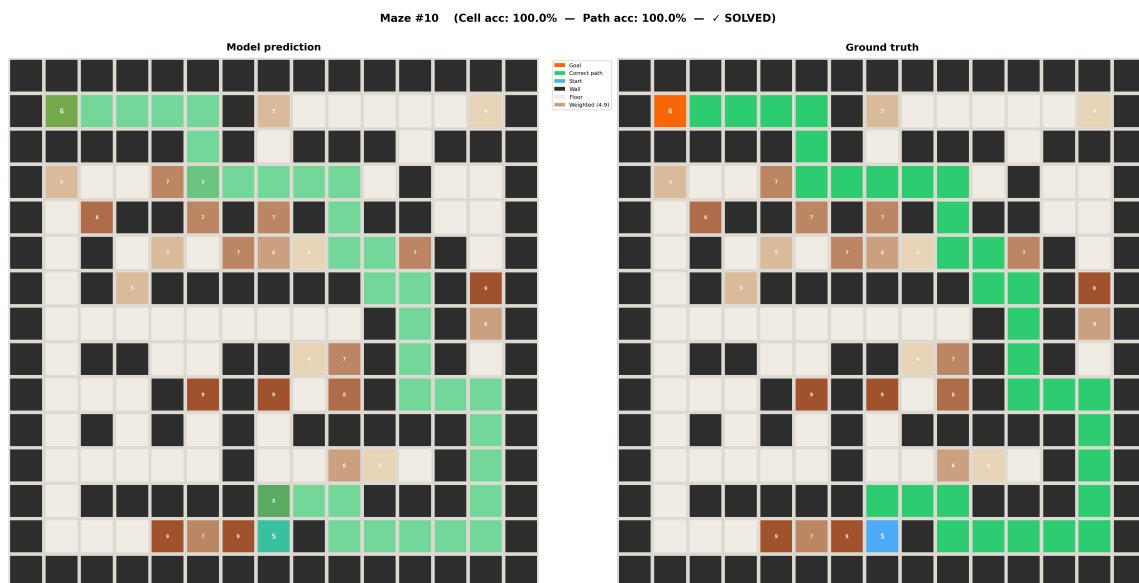


Figure 5.5: Correctly solved 15×15 weighted maze (Maze #10). The model achieves 100% cell accuracy and 100% path accuracy. Left: model prediction. Right: ground truth. The colour legend distinguishes walls (black), floor (white), weighted cells (brown tones), correct path (green), start (blue), and goal (red).

5.3.3 Analysis

The high cell accuracy combined with the lower puzzle accuracy is a characteristic pattern for grid-based prediction tasks: the model correctly classifies the vast majority of cells

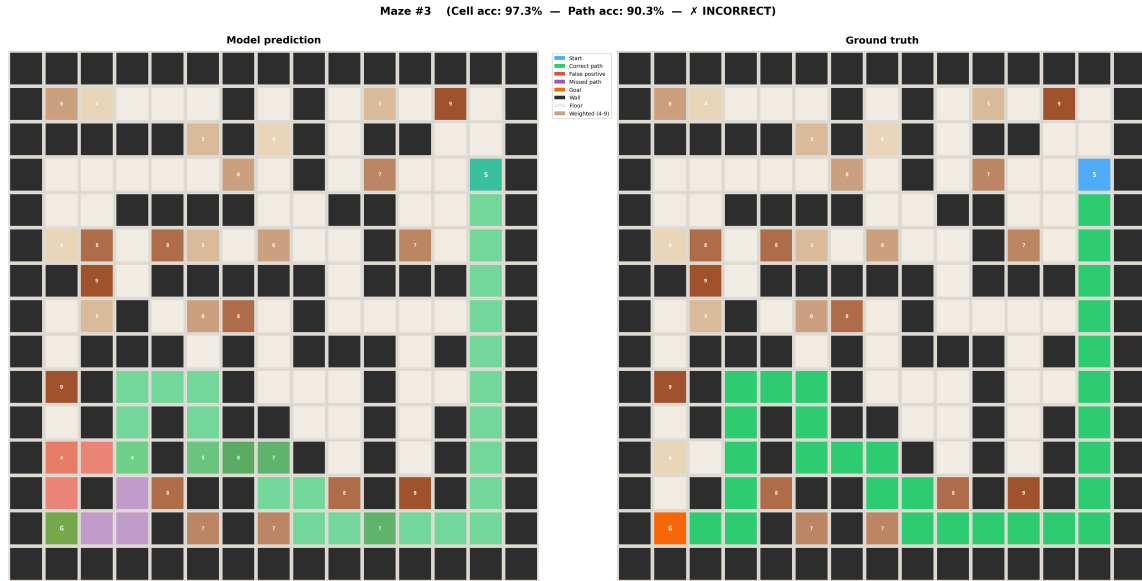


Figure 5.6: Incorrectly solved 15×15 weighted maze (Maze #3). The model achieves 97.3% cell accuracy but only 90.3% path accuracy. Red cells indicate false positives (cells predicted as on-path that are not), and purple cells indicate missed path segments. The errors occur near the bottom of the maze where the model selects a suboptimal route through lower-cost cells.

but occasionally deviates from the optimal path by one or two cells, which is sufficient to count the entire puzzle as incorrect under the strict exact-match criterion. This suggests that the model has learned a strong internal representation of cost-aware pathfinding but occasionally makes local errors at decision points where the optimal path is ambiguous, for example where two routes have similar total costs.

The 75.20% puzzle accuracy on Weighted Maze is a notable result given that this task has not been previously evaluated with any HRM variant. The original Maze-Hard benchmark uses uniform movement costs on 30×30 grids, where the full HRM achieves 74.5% [1]. Direct comparison is complicated by the different grid sizes (15×15 versus 30×30) and the addition of weighted cells, but the result demonstrates that the L-module-only architecture can successfully learn cost-sensitive pathfinding, extending the known capabilities of HRM-style reasoning beyond uniform-cost tasks.

The gap between cell accuracy (over 98%) and puzzle accuracy (75.20%) also suggests that a post-processing step, such as selecting the lowest-cost connected path from the model’s soft predictions, could improve puzzle accuracy without retraining. This would change the evaluation criterion from "model output exactly matches ground truth" to "model output plus post-processing matches ground truth," which is arguably a more practical measure for real applications. This is left as future work.

5.4 Edge Deployment on Raspberry Pi 5

The L-module-only variant was exported to ONNX format and deployed on a Raspberry Pi 5 (16GB RAM, Debian 13 Trixie, 64-bit) using ONNX Runtime with no PyTorch dependency. The export pipeline and zero-dependency inference script are described in Kyrylo Kozlovskyi’s dissertation.



Figure 5.7: Reasoning trajectory for Maze #10 across 16 iterative steps. At Step 1, the model achieves only 86% cell accuracy and 36% path accuracy. By Step 5, it converges to 100% on both metrics, with $\Delta 0$ changed cells in subsequent steps. The visualisation demonstrates how the model progressively refines its solution through iterative latent reasoning, with the bulk of computation occurring in the first four to five steps.

The export was numerically verified to produce outputs identical to the original PyTorch model on the test inputs used during validation, confirming that the conversion process preserves the iterative reasoning semantics despite unrolling the computation loop into a static graph. Accuracy on both Sudoku and Weighted Maze remained unchanged between the PyTorch checkpoint and the ONNX model, as expected when running in matched FP32 precision.

Inference latency was measured using the exported ONNX model with 16 reasoning steps. On the Intel i7-12700H CPU via ONNX Runtime, Sudoku 9×9 inference takes approximately 65 ms per puzzle and Weighted Maze 15×15 inference takes approximately 175 ms per puzzle. On the Raspberry Pi 5, the same tasks take approximately 344 ms

and 1,022 ms respectively, representing a $5.0\text{--}5.8\times$ slowdown relative to the laptop CPU. The Raspberry Pi 5 successfully runs the model in standard FP32 precision without quantisation, confirming that HRM-style iterative reasoning is feasible on modest ARM hardware.

5.5 Cross-Task Comparison

Table 5.4 summarises the L-module-only variant’s performance across both tasks.

Table 5.4: L-module-only performance summary across tasks, averaged across 10 seeds.

Metric	Sudoku	Weighted Maze
Grid Size	9×9	15×15
Puzzle Accuracy	87.82%	75.20%
Cell/Token Accuracy	96.2%	>98%
ONNX CPU Latency (i7-12700H)	≈ 65 ms	≈ 175 ms
ONNX Pi 5 Latency	≈ 344 ms	$\approx 1,022$ ms

The model achieves higher puzzle accuracy on Sudoku than on Weighted Maze. Several factors may explain this difference. Sudoku, while requiring global constraint satisfaction across rows, columns, and 3×3 boxes, benefits from a uniform rule structure: every puzzle follows the same constraints, allowing the model to learn universal patterns that transfer across all instances. The Weighted Maze task, by contrast, requires the model to find the optimal path through a unique spatial layout with heterogeneous costs for each puzzle. Each maze presents a distinct combinatorial search problem where the optimal solution depends entirely on the specific configuration of walls and weights, and the model must effectively learn to approximate Dijkstra’s algorithm rather than apply a fixed rule.

Additionally, the 15×15 maze grid contains 225 cells compared to 81 for Sudoku, presenting a larger prediction space. The strict exact-match criterion is harder to satisfy on larger grids, as there are more cells where an error can occur. Future work could explore whether the accuracy gap narrows on smaller maze grids or widens on larger ones, and whether the model trained on small grids generalises to larger ones at inference time.

5.6 Evaluation Against Objectives

This section evaluates the project against each objective set out in Chapter 1.

Objective 1: Implement a functional Hierarchical Reasoning Model. Achieved. A full HRM with both H-module and L-module has been implemented and exists on a separate branch of the project repository. The model trains successfully and produces predictions, verifying that the architecture has been correctly reproduced from the original paper [1].

Objective 2: Implement an L-module-only variant. Achieved. The L-module-only variant was implemented by removing the H-module while increasing the depth of the L-module, maintaining recurrent behavior, and updating the training script to conform to one-step gradient approximation. This variant served as the primary model for all benchmarking experiments.

Objective 3: Train and benchmark the L-module-only variant on Sudoku puzzles. Achieved. The L-module-only variant was trained and evaluated on mixed-difficulty Sudoku, achieving 87.82% puzzle accuracy averaged across 10 seeds. The secondary goal of benchmarking the full HRM on Sudoku was not completed and is discussed in Section 5.7.

Objective 4: Train and benchmark the L-module-only variant on Weighted Maze. Achieved. The L-module-only variant achieved 75.20% puzzle accuracy and over 98% cell accuracy on 15×15 grids, exceeding the 60% target. The secondary goal of benchmarking the full HRM was not completed.

Edge deployment. Achieved. The L-module-only variant was exported to ONNX and deployed on a Raspberry Pi 5 via ONNX Runtime. Accuracy was preserved across platforms, and inference latency was benchmarked on GPU, CPU, and Pi 5.

5.7 Discussion

5.7.1 Why the L-Module Only Was Prioritised

The decision to focus on the L-module-only variant was guided by two factors. The evidence in the literature discussed in Chapter 3 indicates that it achieves comparable accuracy while training significantly faster, and given limited GPU resources, prioritising it allowed for more thorough experimentation. Its reduced parameter count and simpler forward pass also make it better suited for edge deployment. Running the full HRM benchmarks remains the most immediate item of future work.

5.7.2 Strengths

The project demonstrates several strengths. The Weighted Maze task extends HRM evaluation to cost-sensitive pathfinding, and the 75.20% puzzle accuracy with over 98% cell accuracy provides the first evidence that iterative latent reasoning can handle heterogeneous traversal costs. The same codebase ran on GPU, CPU, and ARM without modification, and the Raspberry Pi 5 deployment confirms that the model is practical on modest hardware without GPU acceleration. The L-module-only results on both tasks are consistent with the view that hierarchical structure may be unnecessary for effective latent reasoning. The trajectory visualisation (Figure 5.7) provides concrete visual evidence of how iterative refinement progressively resolves the solution over multiple reasoning steps.

5.7.3 Limitations

Several limitations should be acknowledged. The absence of full HRM benchmarks is the most significant gap, as it prevents the direct architectural comparison that formed part of the original objectives. The Sudoku evaluation uses a custom-generated mixed-difficulty dataset rather than the published Sudoku-Extreme corpus, which means that comparison with published results requires care, even though the difficulty distribution was designed to closely match Sudoku-Extreme. The Weighted Maze evaluation uses 15×15 grids; testing on larger grids (for example 30×30 to match Maze-Hard) would better demonstrate the model’s scalability. While final accuracy figures were averaged across 10 seeds, individual run variance is not reported with confidence intervals, which would provide a more complete statistical picture. Finally, training time was not formally recorded for

either task, which prevents a direct empirical confirmation of the $2.4\times$ training speedup reported by Ge *et al.* [7].

5.8 Further Benchmarking

The following additional experiments would strengthen the evaluation and are planned as immediate next steps:

- Train and benchmark the full HRM on both Sudoku and Weighted Maze to complete the direct comparison with the L-module-only variant.
- Break down Sudoku accuracy by difficulty tier (easy, medium, hard, expert) to understand where the model fails.
- Report per-seed accuracy with confidence intervals to fully characterise variance.
- Test Weighted Maze on larger grid sizes (for example 20×20 or 30×30) to assess scalability and to test whether a model trained on small grids generalises to larger ones at inference time.
- Investigate the effect of varying the number of reasoning steps on the accuracy of both tasks.
- Record formal training time measurements to empirically validate the Ge *et al.* [7] $2.4\times$ speed-up claim for the L-module-only variant.

Chapter 6

Conclusion

This chapter summarises the project, its findings, and its contributions. It revisits the objectives established in Chapter 1, reflects on the strengths and limitations of the work, and identifies directions for future research.

6.1 Summary

This project implemented, extended, and evaluated the Hierarchical Reasoning Model architecture introduced by Wang *et al.* [1]. Working as a pair, Fionn McCarthy and Kyrylo Kozlovskiy developed an independent, cross-platform HRM implementation in PyTorch, motivated by the architecture’s strong performance on constraint-satisfaction tasks and by evidence from Ge *et al.* [7] that a simplified L-module-only variant may suffice.

The project made four contributions: a cross-platform implementation of both the full HRM and an L-module-only variant; a Weighted Maze task extending HRM evaluation to cost-sensitive pathfinding; training and benchmarking of the L-module-only variant on both tasks; and ONNX export with deployment on a Raspberry Pi 5.

6.2 Key Findings

The L-module-only variant achieved 87.82% puzzle accuracy and 96.2% token accuracy on mixed-difficulty 9×9 Sudoku, averaged across 10 seeds. As discussed in Chapter 5, direct comparison with the original HRM paper’s 55% on Sudoku-Extreme requires care due to differences in difficulty distribution, but the results support the view that computational depth through iterative refinement, rather than hierarchical structure, is the primary driver of reasoning performance.

On the Weighted Maze task, the model achieved 75.20% puzzle accuracy with over 98% cell accuracy on 15×15 grids, averaged across 10 seeds. The high cell accuracy indicates that the model develops a strong internal representation of cost-aware routing, even when strict exact-match puzzle accuracy leaves room for improvement. The contrast with Sudoku is illuminating: Sudoku benefits from a uniform rule structure, whereas each weighted maze presents a unique combinatorial search problem dependent on the specific configuration of walls and weights.

The cross-platform design proved viable: the same codebase trained on GPU and ran inference on CPU and Raspberry Pi 5 without modification. On the Intel i7-12700H CPU

via ONNX Runtime, inference latency was approximately 65 ms per Sudoku puzzle and 175 ms per Weighted Maze puzzle. On the Raspberry Pi 5, the same tasks completed in approximately 344 ms and 1,022 ms respectively, representing a 5.0–5.8 $\times$ slowdown relative to the laptop CPU. Accuracy was preserved across all platforms.

6.3 Objectives Revisited

Objective 1: Implement a functional HRM. Achieved. The full two-module HRM was implemented and verified to train and produce predictions.

Objective 2: Implement an L-module-only variant. Achieved. The H-module was removed while retaining all other components, producing a simplified variant that served as the primary model for evaluation.

Objective 3: Train and benchmark the L-module-only variant on Sudoku. Achieved. The variant achieved 87.82% puzzle accuracy and 96.2% token accuracy averaged across 10 seeds. The secondary goal of benchmarking the full HRM was not completed.

Objective 4: Train and benchmark the L-module-only variant on Weighted Maze. Achieved. The variant achieved 75.20% puzzle accuracy with over 98% cell accuracy on 15 $\times$ 15 grids, exceeding the 60% target. The secondary goal of benchmarking the full HRM was not completed.

Edge deployment. Achieved. The model was exported to ONNX and deployed on a Raspberry Pi 5 via ONNX Runtime, with accuracy preserved across platforms.

6.4 Limitations

The limitations identified in Chapter 5 remain relevant. The most significant is the absence of full HRM benchmarks, which prevents a direct architectural comparison. The custom Sudoku dataset and 15 $\times$ 15 maze grids mean that comparisons with published results on Sudoku-Extreme and the 30 $\times$ 30 Maze-Hard benchmark require care. Training time was not formally recorded, individual run variance is not reported with confidence intervals, and per-difficulty Sudoku breakdowns were not produced.

6.5 Future Work

Several directions for future research emerge from this project.

Complete the architectural comparison. The most immediate next step is to train and benchmark the full HRM on both Sudoku and Weighted Maze under identical conditions to the L-module-only variant, providing a controlled answer to whether hierarchical structure adds value for these tasks.

Quantisation and further edge optimisation. While the Raspberry Pi 5 deployment is now complete in FP32, the interaction between quantisation and fixed-point iteration convergence remains an open question discussed in Chapter 3. Empirical evaluation of INT8 and FP16 quantisation on both accuracy and Pi 5 inference latency would contribute valuable data to this underexplored area.

Scaling the Weighted Maze. Testing on larger grids (20 $\times$ 20, 30 $\times$ 30) with wider weight ranges would reveal how the model’s accuracy scales with task complexity and

whether the L-module-only variant encounters limitations that the full HRM might overcome through hierarchical planning. It would also be interesting to test whether a model trained on small grids generalises to larger ones at inference time.

Difficulty-stratified evaluation. Breaking down Sudoku accuracy by difficulty tier (easy, medium, hard, expert) and Weighted Maze accuracy by grid size or path complexity would provide a more nuanced understanding of the model’s capabilities and failure modes.

Post-processing for path prediction. The high cell accuracy on Weighted Maze (over 98%) suggests that a post-processing step, such as extracting the lowest-cost connected path from the model’s soft output predictions, could improve puzzle accuracy without retraining. This would change the evaluation criterion from exact-match to model output plus post-processing, which is arguably a more practical measure for real applications.

Hybrid architectures. As noted in the literature review, HRM excels at constraint-satisfaction tasks but is poorly suited to tasks requiring world knowledge or natural language understanding. A promising long-term direction is the development of hybrid systems that combine LLM-based semantic reasoning with HRM-style iterative computation for constraint satisfaction, potentially connected through tool-use protocols.

Broader edge device evaluation. Extending deployment to other ARM platforms (NVIDIA Jetson Nano, Apple Silicon via CoreML) would provide a wider picture of HRM edge deployment viability beyond the Raspberry Pi 5.

6.6 Final Remarks

This project provides meaningful contributions to the emerging field of iterative latent reasoning. The Weighted Maze task extends HRM evaluation into cost-sensitive domains, the cross-platform implementation demonstrates that compact reasoning models can run on resource-constrained hardware without architectural compromise, and the results are broadly consistent with the view that computational depth through iterative refinement matters more than hierarchical structure. As research on alternatives to autoregressive token generation continues to develop, the HRM paradigm offers a compelling direction for efficient, compact, and deployable reasoning systems.

GitHub Repository

The full source code, documentation, trained models, and a screencast demonstration for this project are available at:

`https://github.com/fionntmcc/cross-platform-hrm`

The repository contains the complete Python package for both model variants, dataset generators for Sudoku and Weighted Maze, training and evaluation scripts, the CI/CD pipeline configuration, and unit tests.

Bibliography

- [1] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- [2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] Rongwu Xu, Zehan Qi, and Wei Xu. Preemptive answer “attacks” on chain-of-thought reasoning. *arXiv preprint arXiv:2405.20902*, 2024.
- [4] Xinyun Chen et al. Premise order matters in reasoning with large language models. *arXiv preprint arXiv:2402.08939*, 2024.
- [5] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- [6] Jeffrey Seely, Yuki Imajuku, Tianyu Zhao, Edoardo Cetin, and Llion Jones. Sudoku-bench: Evaluating creative reasoning with sudoku variants. *arXiv preprint arXiv:2505.16135*, 2025.
- [7] Renee Ge, Qianli Liao, and Tomaso Poggio. Hierarchical reasoning models: Perspectives and misconceptions. *arXiv preprint arXiv:2510.00355*, 2025.
- [8] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.
- [9] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [10] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [11] Jonas Geiping, Tom Goldstein, et al. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- [12] Tom Dillon. Tdoku: A fast sudoku solver and generator, 2025.
- [13] International Energy Agency. Energy and AI. Technical report, IEA, Paris, 2025.
- [14] Zhengyang Geng, Xin-Yu Zhang, Shaojie Bai, Yisen Wang, and Zhouchen Lin. On training implicit models. *arXiv preprint arXiv:2111.05177*, 2021.

- [15] Timothy P. Lillicrap and Adam Santoro. Backpropagation through time and the brain. *Current Opinion in Neurobiology*, 55:82–89, 2019.
- [16] John D. Murray, Alberto Bernacchia, David J. Freedman, Ranulfo Romo, Jonathan D. Wallis, Xinying Cai, Camillo Padoa-Schioppa, Tatiana Pasternak, Hyojung Seo, Daeyeol Lee, and Xiao-Jing Wang. A hierarchy of intrinsic timescales across primate cortex. *Nature Neuroscience*, 2014.
- [17] Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.
- [18] Yekun Ke, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Advancing the understanding of fixed point iterations in deep neural networks: A detailed analytical study. *arXiv preprint arXiv:2410.11279*, 2024.
- [19] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [20] Pablo Villalobos et al. Will we run out of data? limits of LLM scaling based on human-generated data. *arXiv preprint arXiv:2211.04325*, 2022.
- [21] Mattia Rigotti, Omri Barak, Melissa R. Warden, Xiao-Jing Wang, Nathaniel D. Daw, Earl K. Miller, and Stefano Fusi. The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497:585–590, 2013.
- [22] Valerio Mante, David Sussillo, Krishna V. Shenoy, and William T. Newsome. Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503:78–84, 2013.
- [23] Lorenzo Posani et al. Rarely categorical, always high-dimensional: How the neural code changes along the cortical hierarchy. *bioRxiv*, 2024.
- [24] Pierrick Pochelu. Deep learning inference frameworks benchmark. *arXiv preprint arXiv:2210.04323*, 2022.
- [25] Sandeep Kumar, Sanjay Gupta, and Sanjay Arora. Faster scalable ML model deployment using ONNX. In *IEEE CCWC*, 2021.
- [26] Wenxin Jiang et al. Interoperability in deep learning: A user survey and failure analysis of ONNX model converters. In *ACM ISSTA*, 2024.
- [27] Hyoukjun Kim et al. Extending the ONNX runtime framework for processing-in-memory execution. In *IEEE ISPASS*, 2022.
- [28] Yiming Zhang et al. ONNXPruner: ONNX-based general model pruning adapter. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.
- [29] Wataru Shiraishi et al. ESPnet-ONNX: Bridging a gap between research and production. In *IEEE SLT*, 2022.
- [30] Paul-Edouard Novac et al. Quantization and deployment of deep neural networks on microcontrollers. *Sensors*, 21(9):2984, 2021.

- [31] Hyungjun Ahn, Jiyong Shin, and Jinwon Park. Performance characterization of using quantization for DNN inference on edge devices. *arXiv preprint arXiv:2303.05016*, 2023.
- [32] Xiaoyu Zhao et al. Edge-MPQ: Layer-wise mixed-precision quantization for edge computing. *IEEE Transactions on Computers*, 73(11):2498–2511, 2024.
- [33] Yang Wang et al. Benchmarking edge AI platforms: NVIDIA Jetson and Raspberry Pi 5. In *IEEE ICICT*, 2024.
- [34] Zhenzhen Zhang et al. Feasibility analysis of machine learning optimization on GPU-based low-cost edges. In *IEEE CCIS*, 2021.